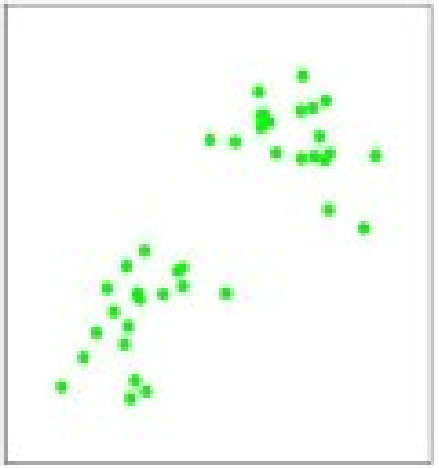
Foundations of Artificial Intelligence

Clustering

Andrea Torsello

- We have assumed we have the correct output labels
  - Supervised learning

- We will now consider the case in which
  - we do not have a training set
  - we want to extract labels from the “structure” of the data



- The problem of finding information from the “structure” of unlabeled data is called
  - Unsupervised learning
  - Clustering

- We are given an unlabeled training set $\{x^{(1)}, \ldots, x^{(m)}\}$
- We want to group the data into a few cohesive clusters.
  - Assume for the moment that
    - the number K of clusters is given
    - The clusters form a partition of the data: data-points are in one and only one cluster

- How do we define cohesiveness?
- Intuitively, we might require that intra-cluster distances are compared with the inter-cluster distances.
- We can formalize this notion by introducing a set of vectors

$$\mu_k \text{ , where } k = 1, \ldots, K$$

- μ_k is a prototype associated with the k th cluster, representing the centers of the clusters.

- Our goal is then to find
  - an assignment of data points to clusters
  - vectors $\{\mu_k\}$,
- such that the sum of the squares of the distances of the data point to the cluster center μ_k , is a minimum.

- Let us introduce a binary indicator variable $r_{nk} \in \{0,1\}$ describing which of the K clusters data point x_n is assigned to.
- We can then define a *distortion measure* as

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$$

K-means algorithm (Lloyd, 1982)

- We optimize J s through an iterative procedure involving two successive steps corresponding to
  - optimization with respect to the r_n
  - optimization with respect to the μ_k .
-
- First we choose some initial values for the μ_k .
  - In the first step Then minimize J with respect to the r_n , keeping the μ_k fixed.
  - In the second step we minimize J with respect to the μ_k , keeping r_n fixed.
- This two-stage optimization is repeated until convergence.

Optimization of r_{nk}

- Since J is linear in r_{nk}, we can give minimum in a closed form solution by setting $r_{nk}=1$ for whichever value of k gives the minimum of $\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$

$$r_{nk} = \begin{cases} 1 & \text{if } k = \arg\min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0 & \text{otherwise.} \end{cases}$$

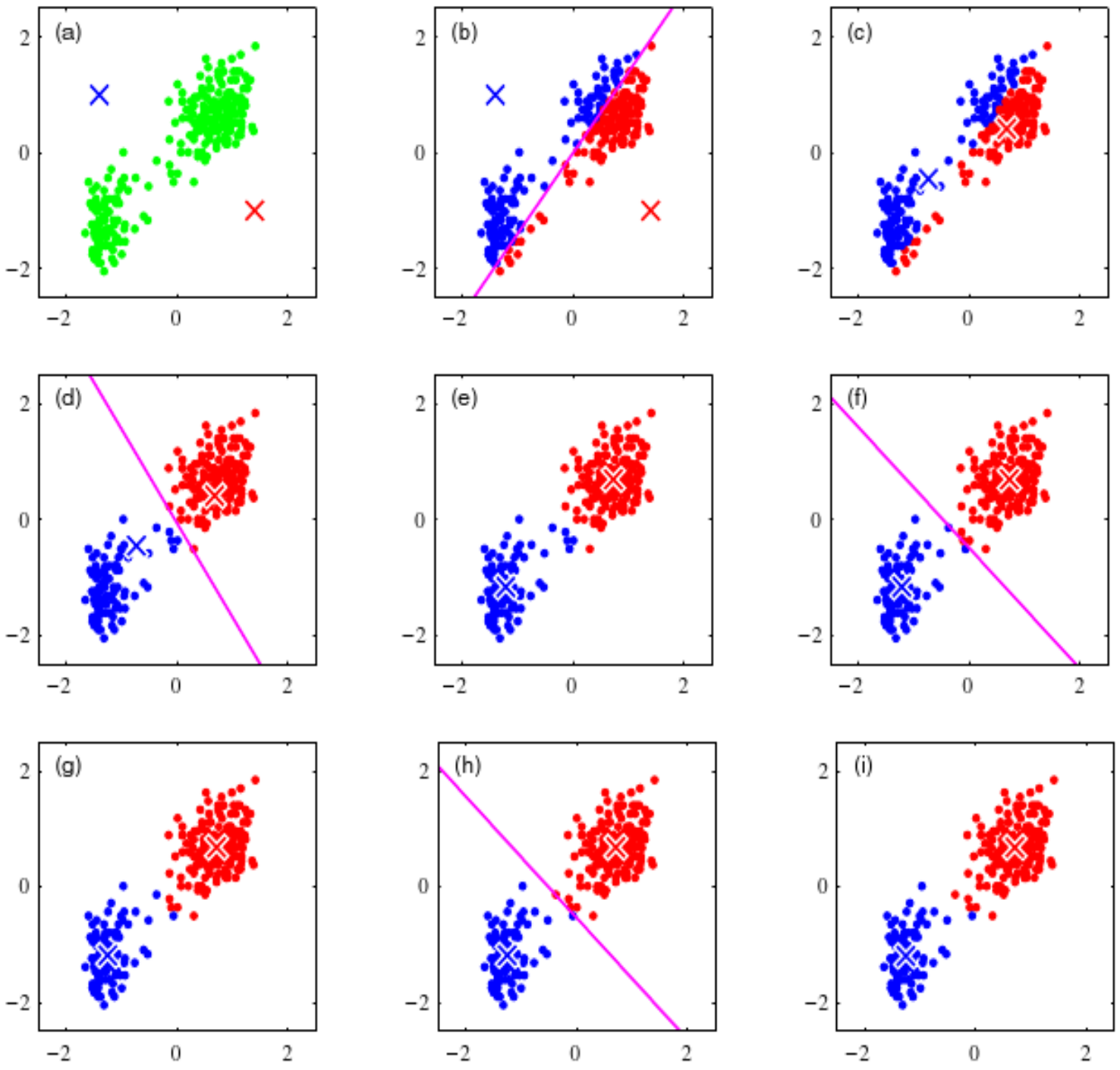
Optimization of μ_k

- Function J is quadratic in μ_k, and it can be minimized by setting its derivative with respect to μ_k to zero giving

$$2\sum_{n=1}^{N} r_{nk}(\mathbf{x}_n - \boldsymbol{\mu}_k) = 0 \qquad \boldsymbol{\mu}_k = \frac{\sum_n r_{nk}\mathbf{x}_n}{\sum_n r_{nk}}.$$

- This sets μ_k equal to the mean of all of the data points $\mathbf{x}_n$ assigned to cluster k, hence the name K-means algorithm.

- The two phases are repeated in turn until there is no further change in the assignments (or until some maximum number of iterations is exceeded).


(a)
(b)
(c)
(d)
(e)
(f)
(g)
(h)
(i)
2
0
−2
−2
0
2


$K = 2$
$K = 3$
$K = 10$
Original image

- K-means is a descent algorithm (each phase reduces J)
  - Convergence is assured (almost).
  - Might converge to a local minimum

- The K-means algorithm is based on the use of squared Euclidean distance
  - this limits the type of data (no categorical labels for instance)
  - nonrobust to outliers.

- We can generalize introducing a more general dissimilarity V(**x**,**x**')

$$\widetilde{J} = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \mathcal{V}(\mathbf{x}_n, \boldsymbol{\mu}_k)$$

- Hard to optimize the μ_k => limit each centroid to be equal to one data point.
- K-medoids algorithm

- At each iteration, every data point is assigned uniquely to one, and only one, of the clusters.
- For data points that lie roughly midway between cluster centres, it is not clear that the hard assignment to the nearest cluster is the most appropriate.
- Adopting a probabilistic approach, we obtain 'soft' assignments of data points to clusters in a way that reflects the level of uncertainty over the most appropriate assignment.
- This probabilistic formulation brings with it numerous benefits.

Gaussian Mixture

- A mixture of n random variables $\mathbf{X}_i$ according to the multinomial mixture $\boldsymbol{\pi}$ is a random variable **Y** that samples data-points according to the following rule:
  - Sample index *k* from $\boldsymbol{\pi}$ and then sample the point from $\mathbf{X}_k$
- It is easy to see that the density of the mixture Y is

$$d_{\mathbf{Y}}(\mathbf{x}) = \sum_{i=1}^{n} \pi_i d_{\mathbf{X}_i}(\mathbf{x})$$

where $d_{\mathbf{X}}(\mathbf{x})$ is the density of $\mathbf{X}_i$ and π_i=P{$\boldsymbol{\pi}$=i}

- Let us model the observation as a mixture of K Gaussians

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- Let **z** be a binary indicator variable
- We can define the joint distribution p(**x**, **z**) in terms of
  - marginal distribution p(**z**)
  - conditional distribution p(**x**|**z**)

- p(**z**) is specified in terms of the mixing coefficients π_i

$$p(z_k = 1) = \pi_k$$

- This can be written in the form

$$p(\mathbf{z}) = \prod_{k=1}^{K} \pi_k^{z_k}.$$

- Similarly, p(**x**|**z**) is a Gaussian

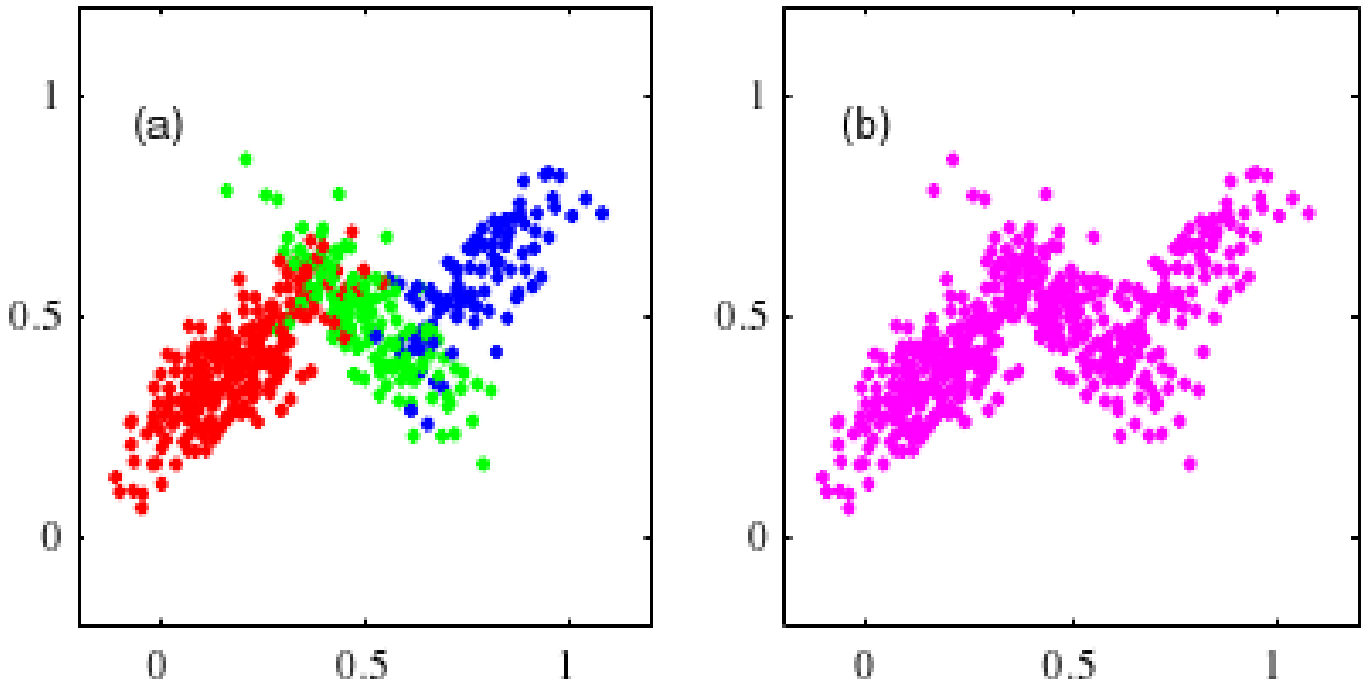
$$p(\mathbf{x}|z_k = 1) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- Or,

$$p(\mathbf{x}|\mathbf{z}) = \prod_{k=1}^{K} \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_k}$$

- The marginal distribution p(**x**) can be obtained by summing p(**x**, **z**) over all the possible states **z**

$$p(\mathbf{x}) = \sum_{\mathbf{z}} p(\mathbf{z})p(\mathbf{x}|\mathbf{z}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$


(a)
(b)
1
0.5
0
0
0.5
1

- We have therefore found an equivalent formulation of the Gaussian mixture involving an explicit latent variable **z**.
- Clustering is thus reduced to the estimation of **z**

- It might seem that we have not gained much by adding **z**
- However, we are now able to work with the joint distribution p(x, z) which will allow us to use the the **expectation-maximization (EM) algorithm**

Expectation Maximization (EM) Algorithm

- The EM algorithm (Dempster et al., 1977; McLachlan and Krishnan, 1997)
- A general technique for finding maximum likelihood estimates for probabilistic models with latent variables

- Let $\boldsymbol{X}$ be the observed variables
  - Row n correspond to data point $\boldsymbol{x}_n^{\mathsf{T}}$
- Let $\boldsymbol{Z}$ be the latent variables
- The joint distribution $p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\theta})$ is governed by a set of parameters θ
- The likelihood is

$$p(\mathbf{X}|\boldsymbol{\theta}) = \sum_{\mathbf{Z}} p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\theta})$$

- Assume
  - Direct optimization of $p(\mathbf{X}|\boldsymbol{\theta})$ is hard
  - If you could observe $\boldsymbol{Z}$, optimizing $p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\theta})$ would be easy

- Let $q(\mathbf{Z})$ be any distribution over $\mathbf{Z}$
- Define

$$\mathcal{L}(q, \boldsymbol{\theta}) = \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \left\{ \frac{p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\theta})}{q(\mathbf{Z})} \right\}$$

- We have

$$\ln p(\mathbf{X}|\boldsymbol{\theta}) = \mathcal{L}(q, \boldsymbol{\theta}) + \mathrm{KL}(q\|p)$$

- With

$$\mathrm{KL}(q\|p) = -\sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \left\{ \frac{p(\mathbf{Z}|\mathbf{X}, \boldsymbol{\theta})}{q(\mathbf{Z})} \right\}$$

- In fact $p(\mathbf{X}, \mathbf{Z}|\theta) = p(\mathbf{Z}|\mathbf{X}, \theta)p(\mathbf{X}|\theta)$
- thus $\mathcal{L}(q, \theta) + \mathrm{KL}(q||p) = \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \left\{ \frac{p(\mathbf{X}, \mathbf{Z}|\theta)}{q(\mathbf{Z})} \frac{q(\mathbf{Z})}{p(\mathbf{Z}|\mathbf{X}, \theta)} \right\} =$

$$\sum_{\mathbf{Z}} q(\mathbf{Z}) \ln p(\mathbf{X}|\theta) = \ln p(\mathbf{X}|\theta)$$

- Remember that $\mathrm{KL}(q||p) \geq 0$ with equality iff *q=p*
  - i.e., iff q is the posterior $p(\mathbf{Z}|\mathbf{X}, \theta)$ of the latent variable ***Z***

- Thus $\mathcal{L}(q, \boldsymbol{\theta}) \leqslant \ln p(\mathbf{X}|\boldsymbol{\theta})$

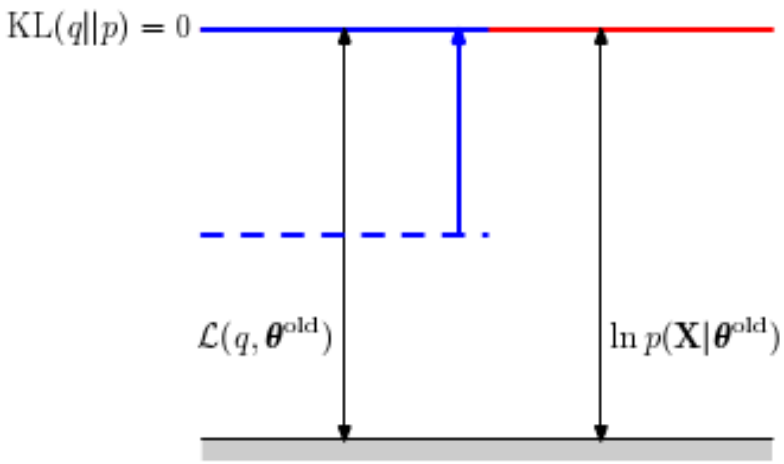
- The EM algorithm is a two-stage iterative optimization technique for finding maximum likelihood solutions.

- (E step) the lower bound $\mathcal{L}(q, \theta)$ is maximized with respect to
  - the **distribution** *q(**Z**)* of the latent variable ***Z***
  - keeping fixed the parameters **θ**
    - The optimum occurs for $q(\mathbf{Z}) = p(\mathbf{Z}|\mathbf{X}, \theta)$

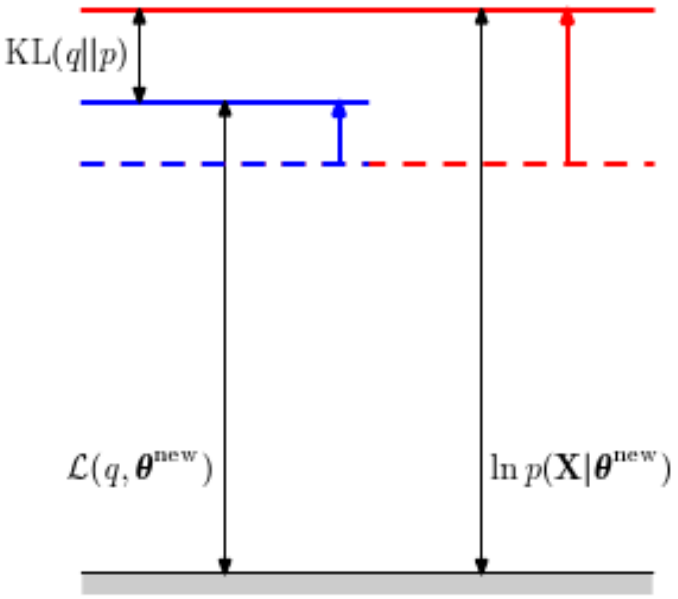
- (M step) $\mathcal{L}(q, \theta)$ is maximized with respect to the parameters **θ** keeping fixed *q(**Z**)*

- *q(**Z**)* it will not equal the new posterior
  - there will be a nonzero KL divergence.

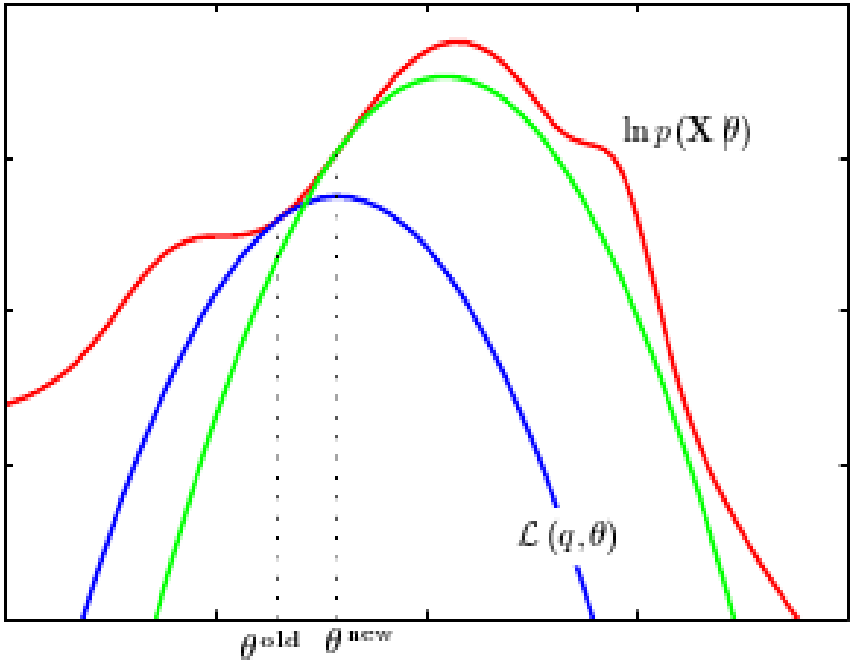
- The increase in $\ln p(\mathbf{X}|\theta)$ is therefore greater than the increase in $\mathcal{L}(q, \theta)$


$\mathrm{KL}(q||p) = 0$
$\mathcal{L}(q, \boldsymbol{\theta}^{\mathrm{old}})$
$\ln p(\mathbf{X}|\boldsymbol{\theta}^{\mathrm{old}})$

E step


$\mathrm{KL}(q||p)$
$\mathcal{L}(q, \boldsymbol{\theta}^{\mathrm{new}})$
$\ln p(\mathbf{X}|\boldsymbol{\theta}^{\mathrm{new}})$

M step


$\ln p(\mathbf{X}|\theta)$
$\mathcal{L}(q, \theta)$
θ^{old}
θ^{new}

Mixture of Gaussians

- Let us apply the EM algorithm to the mixture of Gaussians.

- Represent with $\gamma(z_{nk})=p(z_{nk}=1|\boldsymbol{x},\theta)$ the current distribution of $\boldsymbol{Z}$
  - γ Is a NxK matrix with row-sums equal to 1

- The E step updates γ setting it to the posterior of $\boldsymbol{Z}$

$$\gamma(z_{nk}) = p(z_{nk} = 1|\mathbf{x}_n, \mu_k, \mathbf{\Sigma}_k) = p(\mathbf{x}_n|z_{nk} = 1, \mu_k, \mathbf{\Sigma}_k)\frac{p(z_{nk} = 1|\mu_k, \mathbf{\Sigma}_k)}{p(\mathbf{x}_n|\mu_k, \mathbf{\Sigma}_k)}$$

- But $p(z_{nk}=1|\mu_k, \mathbf{\Sigma}_k)=\pi_k$ and $p(\mathbf{x}_n|\mu_k, \mathbf{\Sigma}_k) = \sum_z p(\mathbf{x}_n|z, \mu_k, \mathbf{\Sigma}_k)p(z|\mu_k, \mathbf{\Sigma}_k)$

  thus

$$\gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n|\mu_k, \mathbf{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\mathbf{x}_n|\mu_j, \mathbf{\Sigma}_j)}$$

- In the M step we optimize $\boldsymbol{\mu}_k$, $\boldsymbol{\Sigma}_k$, and $\boldsymbol{\pi}_k$
- We have $\mathcal{L} = \sum_{n=1}^{N}\sum_{k=1}^{K}\gamma(z_{nk})\ln\left\{\frac{p(\mathbf{x}_n, z_{nk}|\mu_k, \boldsymbol{\Sigma}_\mathbf{k})}{\gamma(z_{nk})}\right\} =$

$$\sum_{n=1}^{N}\sum_{k=1}^{K}\gamma(z_{nk})\left(\ln\pi_k - \frac{1}{2}(\mathbf{x}_n-\mu_k)^T\boldsymbol{\Sigma}_k(\mathbf{x}_n-\mu_k) + \text{const}\right)$$

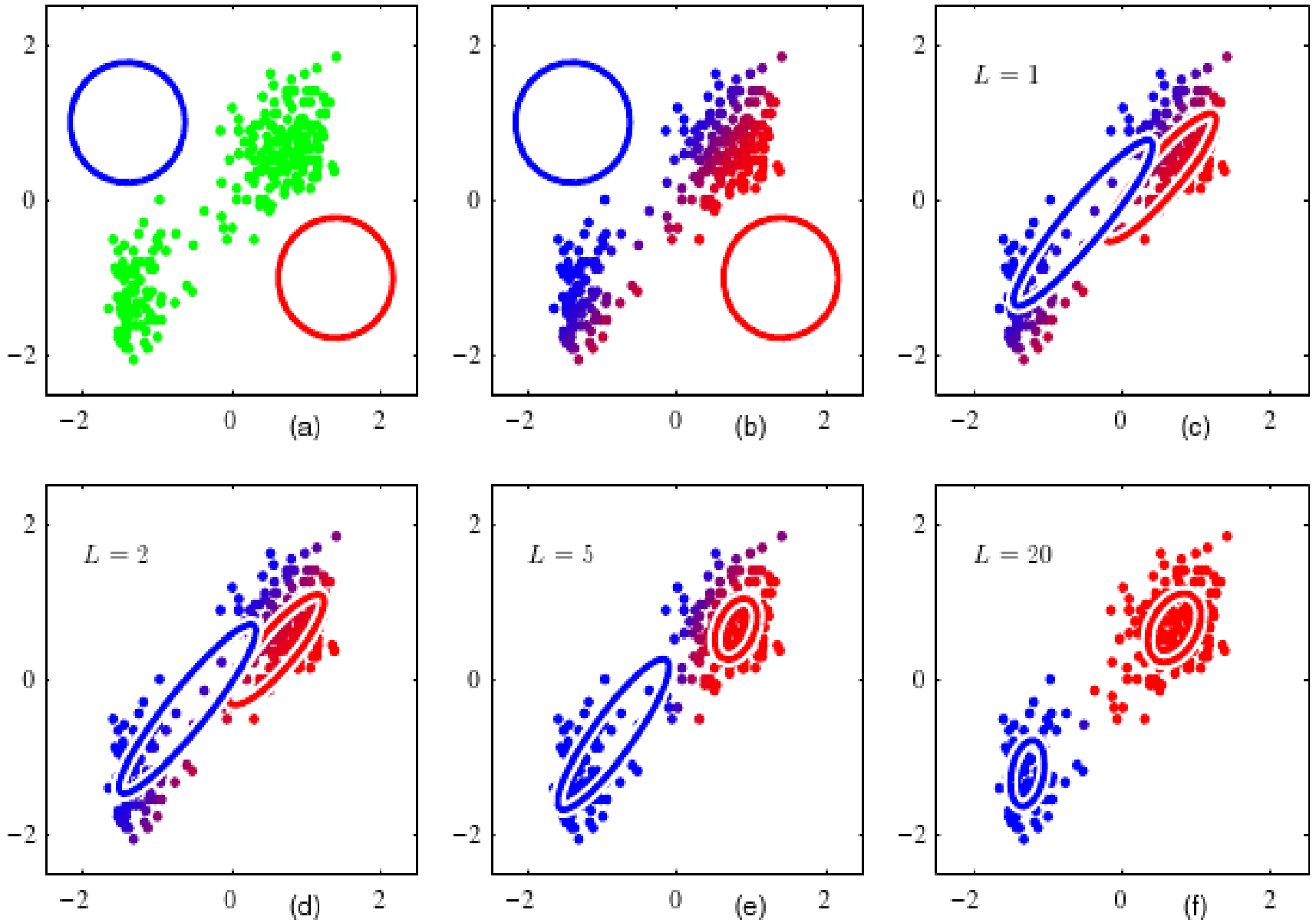
- Setting the derivatives to 0 and recalling that $\sum_{k=1}^{K}\pi_k = 1$ we have

$$\boldsymbol{\mu}_k = \frac{1}{N_k}\sum_{n=1}^{N}\gamma(z_{nk})\mathbf{x}_n \qquad \text{with} \qquad N_k = \sum_{n=1}^{N}\gamma(z_{nk}).$$

and

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k}\sum_{n=1}^{N}\gamma(z_{nk})(\mathbf{x}_n-\boldsymbol{\mu}_k)(\mathbf{x}_n-\boldsymbol{\mu}_k)^{\mathrm{T}}$$

$$\pi_k = \frac{N_k}{N}$$


2
0
−2
−2
0
2
(a)
(b)
$L = 1$
(c)
$L = 2$
(d)
$L = 5$
(e)
$L = 20$
(f)

Relation with k-means

- K-means and mixture of Gaussian have several features in common
  - Both require a latent indicator variable
  - Both are coordinate ascent algorithms
  - Both maximize the objective iteratively
    - on the latent variables
    - then on the model parameters

- In fact k-means can be seen as a limit case of EM for a Gaussian mixture model

- Consider a mixture model of K Gaussian components with fixed covariance matrix $\epsilon\mathbf{I}$, for some (small) value ϵ

- We have $$p(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi\epsilon)^{1/2}} \exp\left\{-\frac{1}{2\epsilon}\|\mathbf{x} - \boldsymbol{\mu}_k\|^2\right\}$$

- The posterior of the latent variable is

$$\gamma(z_{nk}) = \frac{\pi_k \exp\left\{-\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2/2\epsilon\right\}}{\sum_j \pi_j \exp\left\{-\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2/2\epsilon\right\}}$$

Relation with k-means

- If we consider the limit for $\epsilon \to 0$, the term for which $||\mathbf{x}_n - \mu_j||^2$ is smallest will go to 0 most slowly
  - All responsibilities $\gamma(z_{nk})$ for the point $\boldsymbol{x}_n$ will go to 0 except for term j
  - $\gamma(z_{nJk}) \to 1$
  - Independently of $\boldsymbol{\pi}$, as long as it has no 0 terms

- In this limit, assignments are hard!
  - $\gamma(z_{nJk}) \to r_{nk}$
  - And M-step is equivalent to k-means's parameter re-estimation

- Finally, $$\mathbb{E}_{\mathbf{Z}}[\ln p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi})] \to -\frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K} r_{nk}\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2 + \text{const.}$$

Thus maximizing the expected complete-data likelihood is equivalent to minimizing the distortion J

Mixture of Bernoulli distributions (latent class analysis)

- Let us see another important example of mixture model estimated through the EM algorithm
- Consider D *independent* binary variable x_i, i=1,...,D
  - Each governed by a Bernoulli distribution with parameter μ_i

$$p(\mathbf{x}|\boldsymbol{\mu}) = \prod_{i=1}^{D} \mu_i^{x_i}(1-\mu_i)^{(1-x_i)}$$

- Mean and covariance are

$$\begin{aligned} \mathbb{E}[\mathbf{x}] &= \boldsymbol{\mu} \\ \mathrm{cov}[\mathbf{x}] &= \mathrm{diag}\{\mu_i(1-\mu_i)\}. \end{aligned}$$

- Consider a mixture of K component with proportions $\boldsymbol{\pi}$

$$p(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\pi}) = \sum_{k=1}^{K} \pi_k p(\mathbf{x}|\boldsymbol{\mu}_k) \qquad p(\mathbf{x}|\boldsymbol{\mu}_k) = \prod_{i=1}^{D} \mu_{ki}^{x_i}(1-\mu_{ki})^{(1-x_i)}$$

- Mixing the Bernoulli model allows for element correlation
  - Mixture-conditional independence is equivalent to class-conditional independence of naïve Bayes
  - The only difference is that mixture membership is not given in the training set

- The mean and covariance of the mixture can be easily computed

$$\mathbb{E}[\mathbf{x}] = \sum_{k=1}^{K} \pi_k \boldsymbol{\mu}_k$$

$$\operatorname{cov}[\mathbf{x}] = \sum_{k=1}^{K} \pi_k \left\{ \boldsymbol{\Sigma}_k + \boldsymbol{\mu}_k \boldsymbol{\mu}_k^{\mathrm{T}} \right\} - \mathbb{E}[\mathbf{x}]\mathbb{E}[\mathbf{x}]^{\mathrm{T}}$$

with $\boldsymbol{\Sigma}_k = \operatorname{diag}\{\mu_{ki}(1 - \mu_{ki})\}$

- The log-likelihood of this model is

$$\ln p(\mathbf{X}|\boldsymbol{\mu}, \boldsymbol{\pi}) = \sum_{n=1}^{N} \ln \left\{ \sum_{k=1}^{K} \pi_k p(\mathbf{x}_n|\boldsymbol{\mu}_k) \right\}$$

- Summation inside of the logarithm: no closed form solution!

- Let us add a latent variable $\boldsymbol{z}=(z_1,...,z_K)^\top$ associating data to mixture components
  - The conditional distribution of $\boldsymbol{x}$, given $\boldsymbol{z}$, is

$$p(\mathbf{x}|\mathbf{z}, \boldsymbol{\mu}) = \prod_{k=1}^{K} p(\mathbf{x}|\boldsymbol{\mu}_k)^{z_k}$$

  - While the prior of $\boldsymbol{z}$ is $\quad p(\mathbf{z}|\boldsymbol{\pi}) = \prod_{k=1}^{K} \pi_k^{z_k}$

- The complete-data log-likelihood is, thus,

$$\ln p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\mu}, \pi) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \left\{ \ln \pi_l + \sum_{i=1}^{D} \left[x_{ni} \ln \mu_{ki} + (1 - x_{ni}) \ln(1 - \mu_{ki}) \right] \right\}$$

where $\boldsymbol{X}=\{\boldsymbol{x}_n\}$ and $\boldsymbol{Z}=\{\boldsymbol{z}_n\}$

- The E-step maximizes the expected log-likelihood with respect to the latent-variable distribution γ assigning it to the posterior

$$\gamma(z_{nk}) = \mathbb{E}[z_{nk}] = \frac{\sum_{z_{nk}} z_{nk} \left[\pi_k p(\mathbf{x}_n|\boldsymbol{\mu}_k) \right]^{z_{nk}}}{\sum_{z_{nj}} \left[\pi_j p(\mathbf{x}_n|\boldsymbol{\mu}_j) \right]^{z_{nj}}} = \frac{\pi_k p(\mathbf{x}_n|\boldsymbol{\mu}_k)}{\sum_{j=1}^{K} \pi_j p(\mathbf{x}_n|\boldsymbol{\mu}_j)}$$

- The M-step maximizes the expected log-likelihood

$$\mathbb{E}_{\mathbf{Z}}[\ln p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\mu}, \boldsymbol{\pi})] = \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma(z_{nk}) \left\{ \ln \pi_k + \sum_{i=1}^{D} [x_{ni} \ln \mu_{ki} + (1 - x_{ni}) \ln(1 - \mu_{ki})] \right\}$$

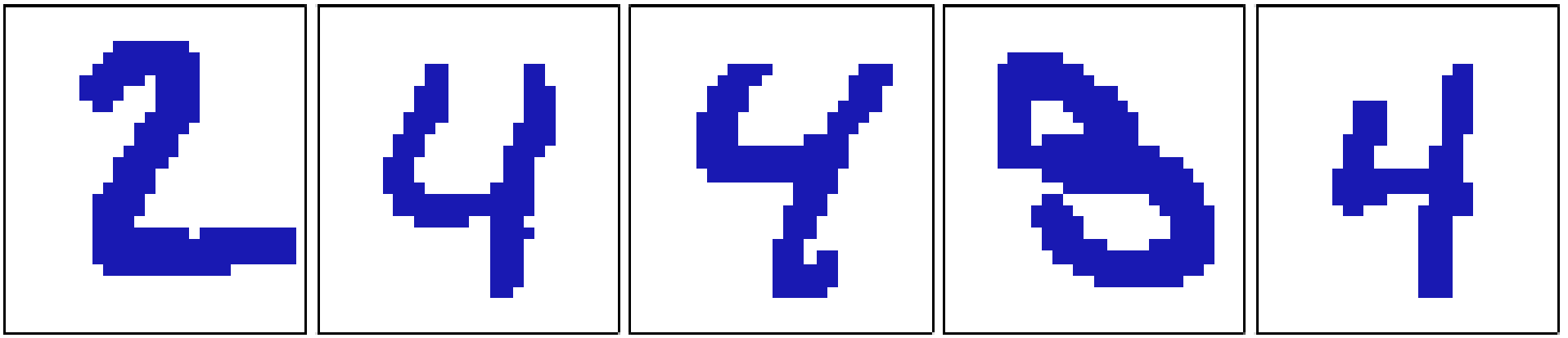
  with respect to the parameters **μ** and **π**

-
- The optimizers are $\boldsymbol{\mu}_k = \overline{\mathbf{x}}_k$ and $\pi_k = \frac{N_k}{N}$

  where

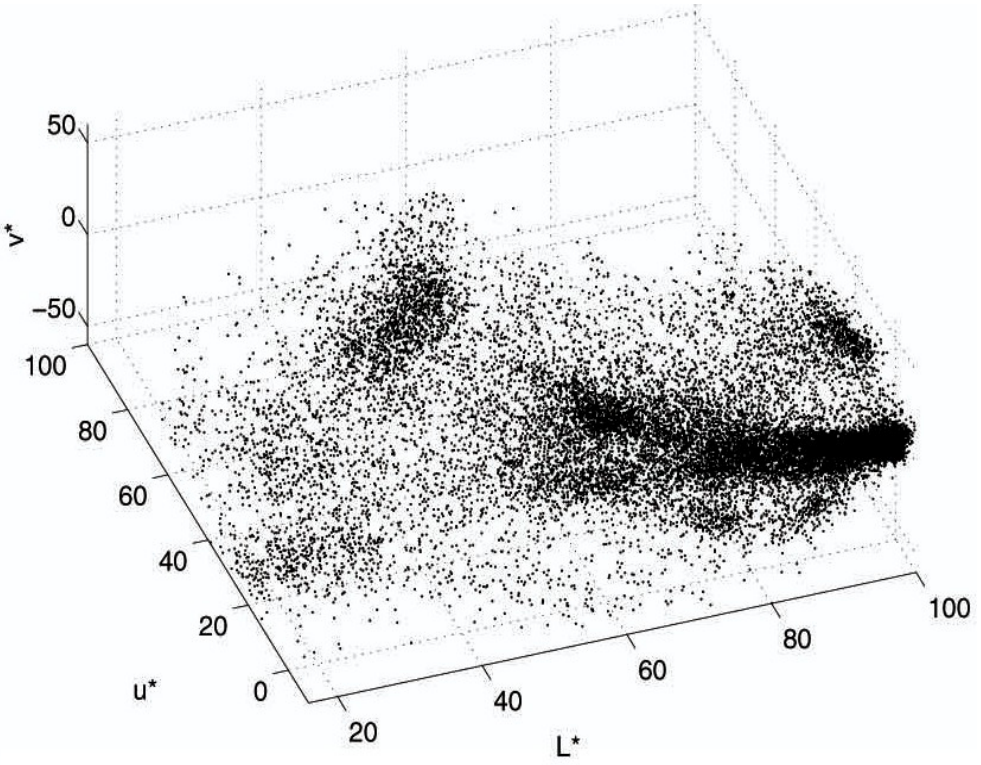
$$N_k = \sum_{n=1}^{N} \gamma(z_{nk})$$

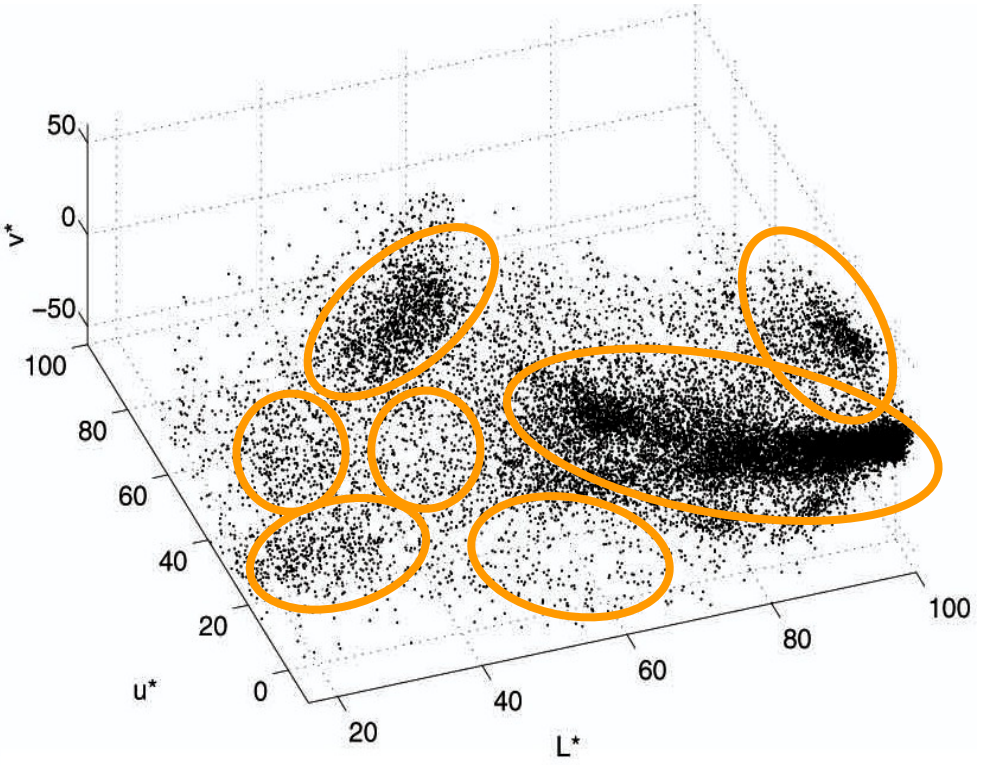
$$\mathbf{x}_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk}) \mathbf{x}_n$$



Mean Shift

A non-parametric technique for analyzing complex multimodal feature spaces and estimating the stationary points (modes) of the underlying probability density function ***without explicitly estimating it***.


50
0
−50
v*
100
80
60
40
20
0
u*
20
40
60
80
100
L*

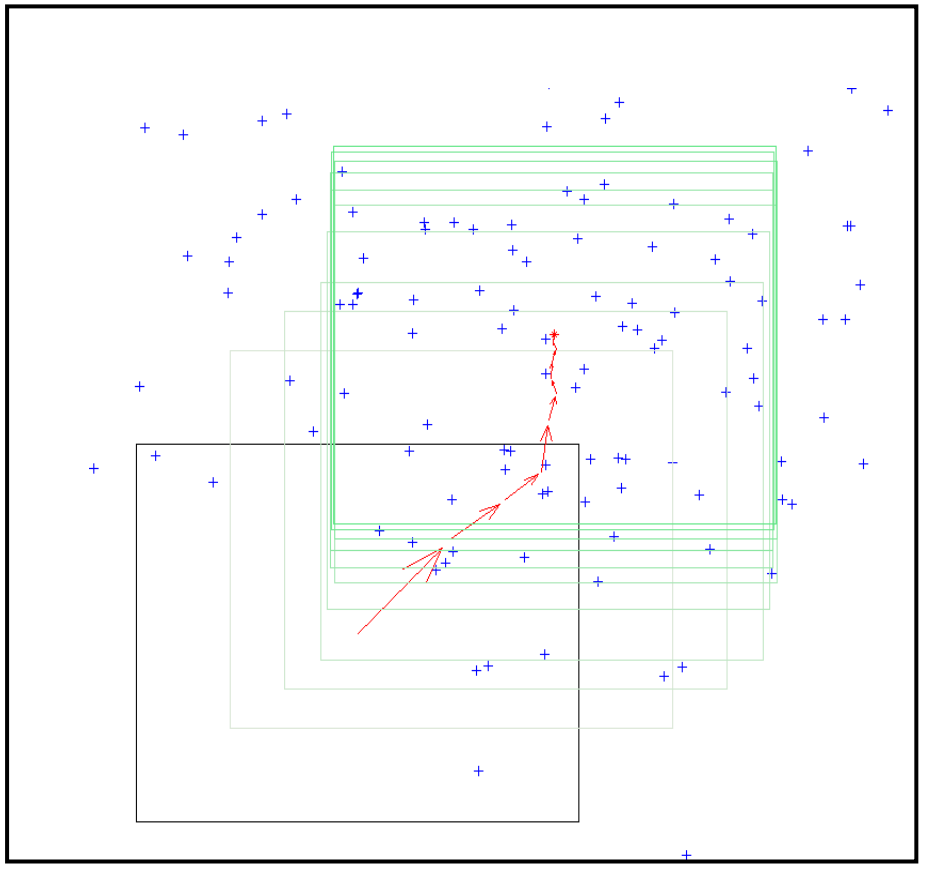


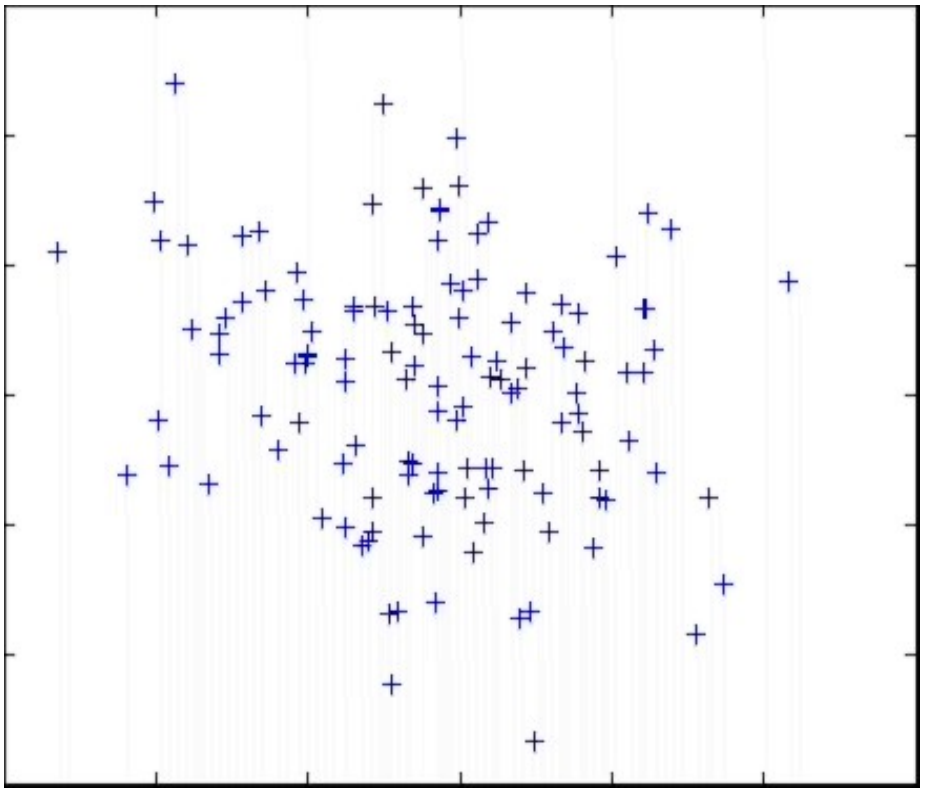
Parametric Density Estimation?

Mean Shift Algorithm

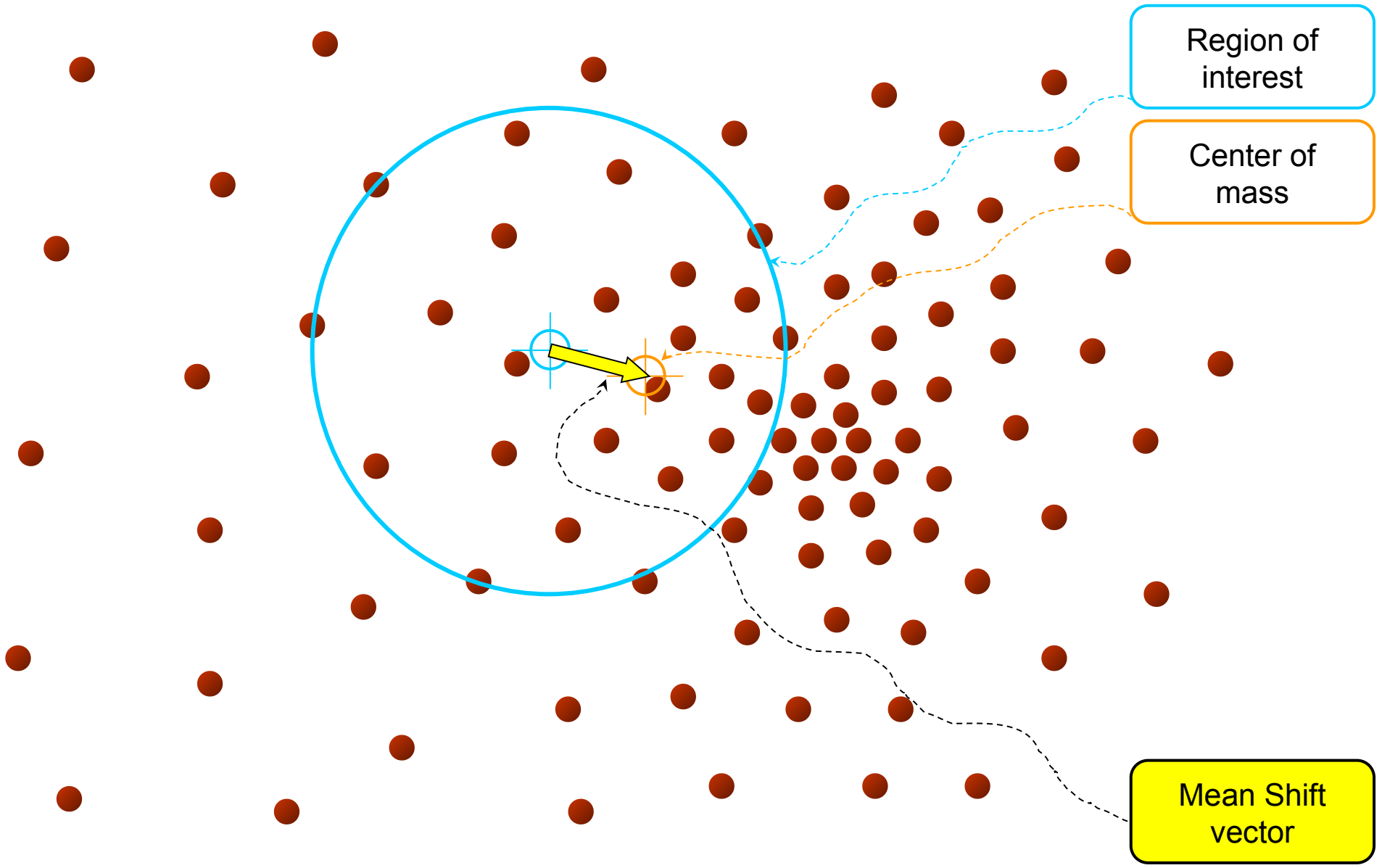
1. Choose a search window size.
2. Choose the initial location of the search window.
3. Compute the mean location (centroid of the data) in the search window.
4. Center the search window at the mean location computed in Step 3.
5. Repeat Steps 3 and 4 until convergence.

The mean shift algorithm seeks the “mode” or point of highest density of a data distribution:



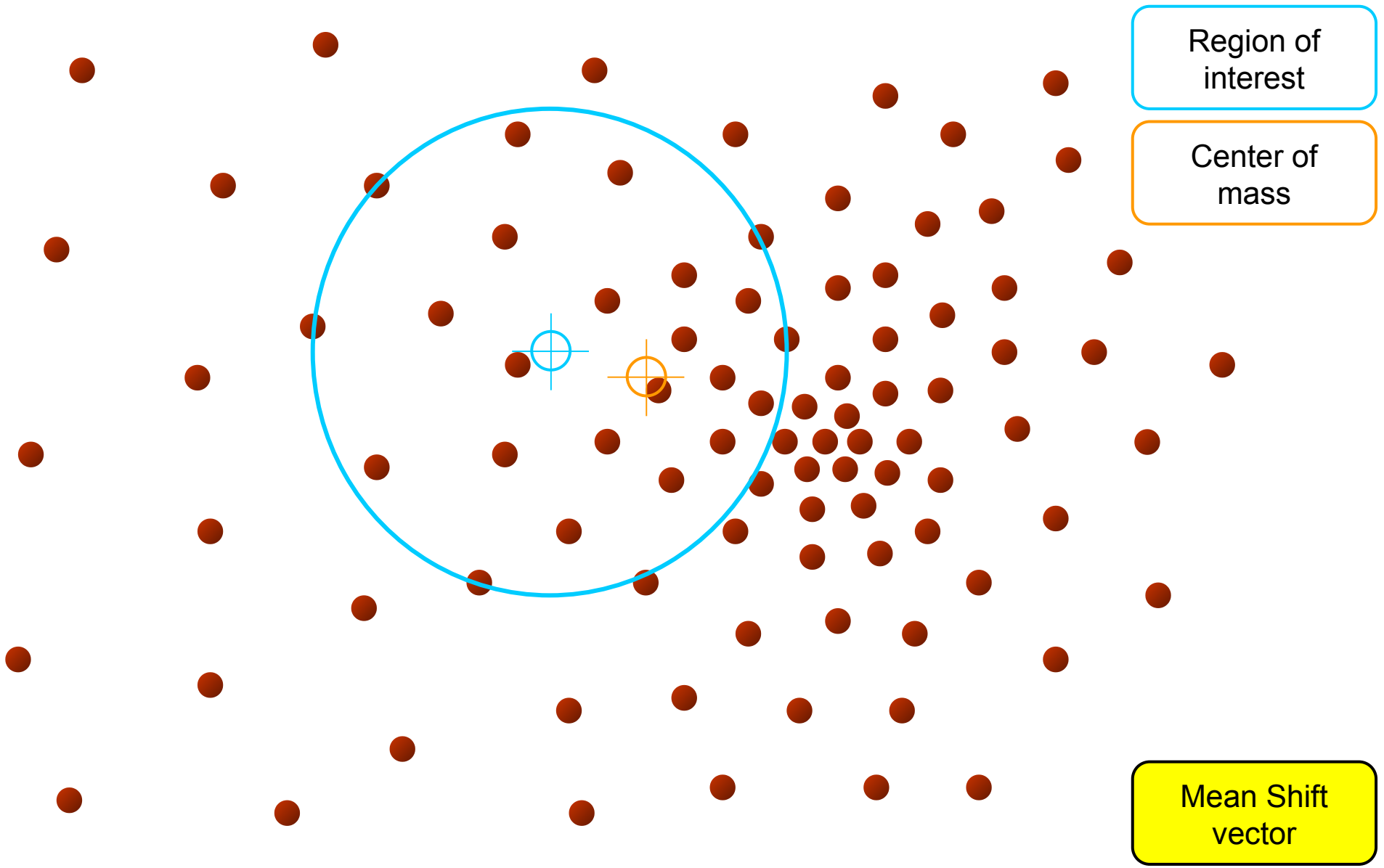


Intuitive Description



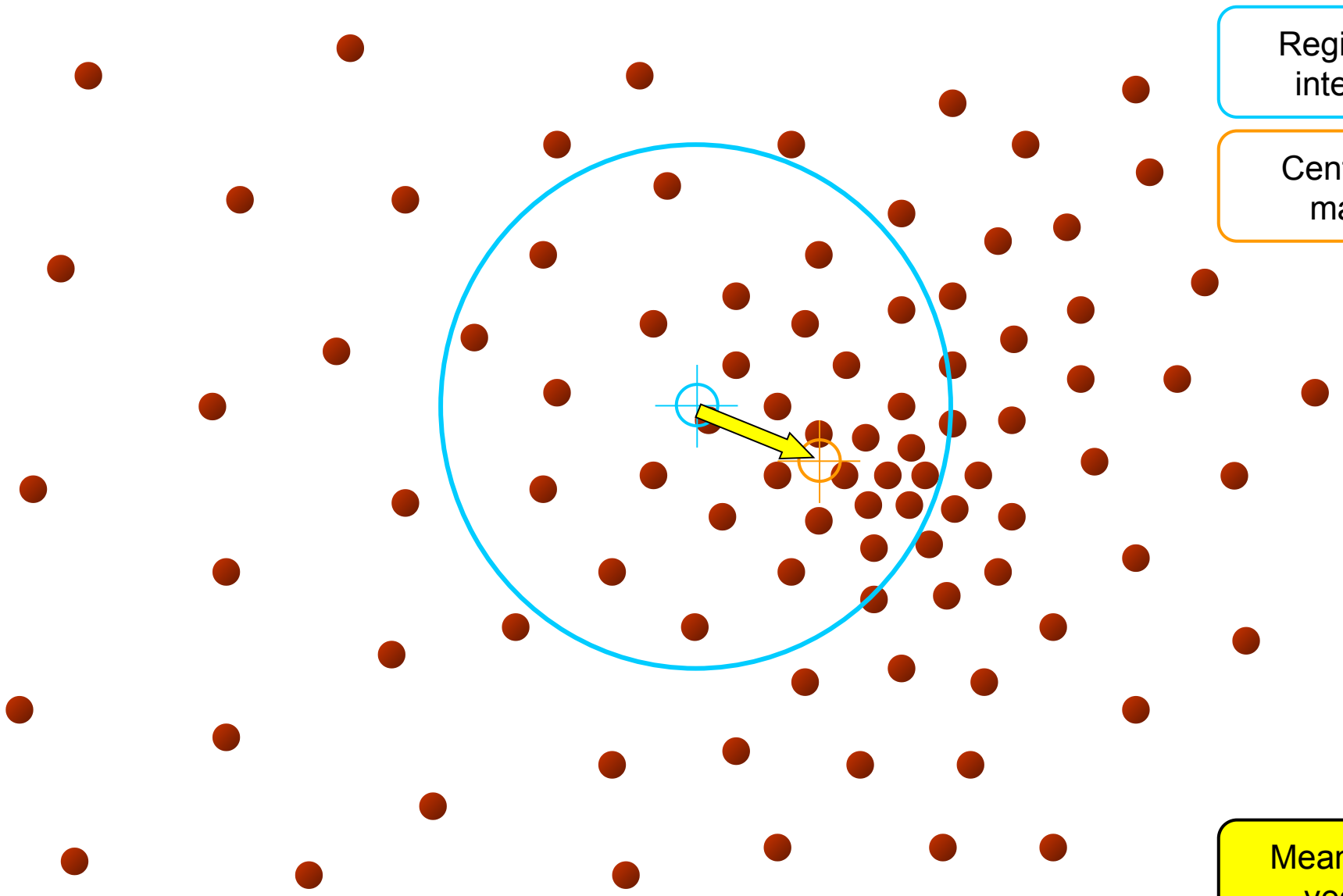
Objective : Find the densest region

Intuitive Description



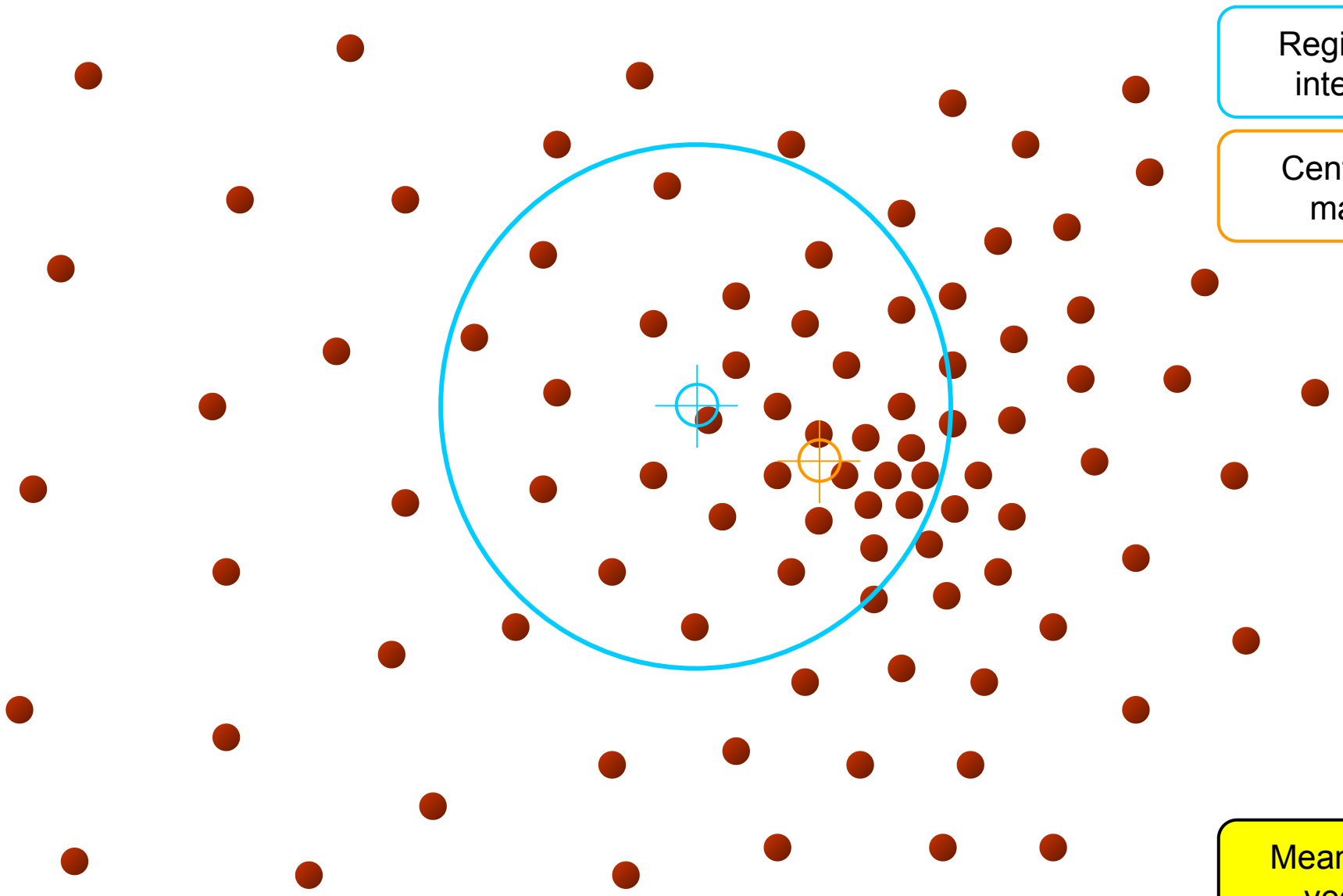
Objective : Find the densest region

Intuitive Description



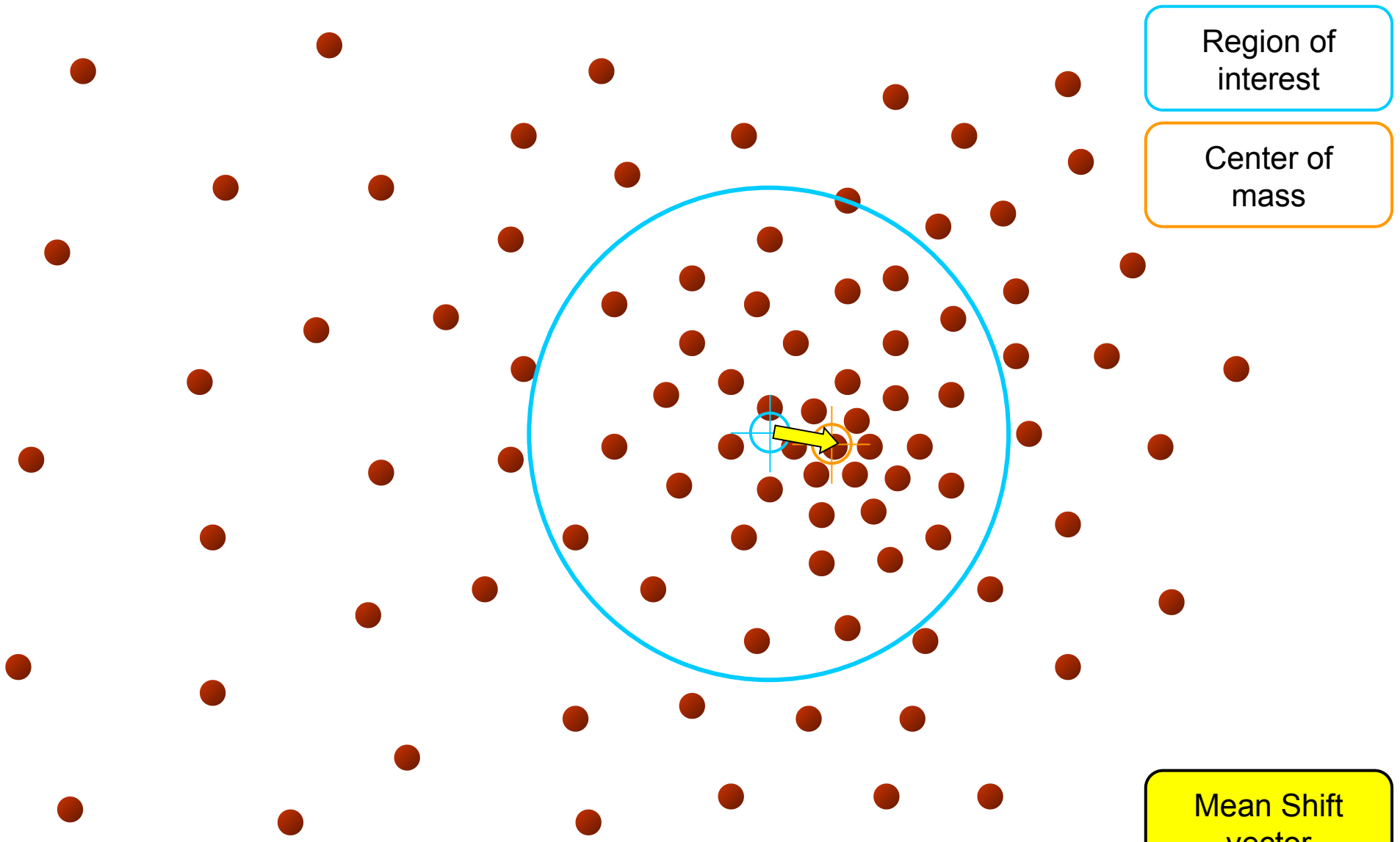
Objective : Find the densest region

Intuitive Description



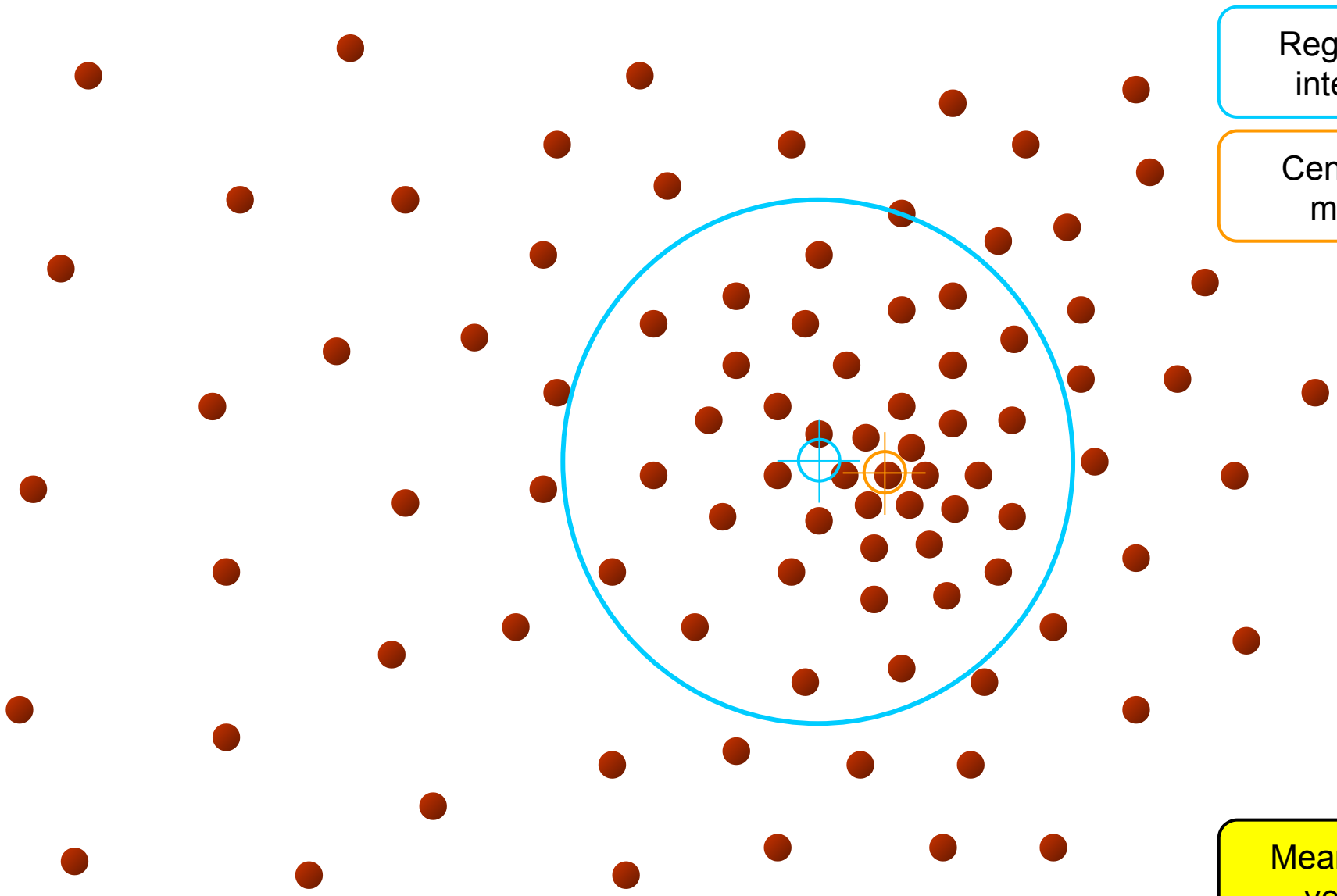
Objective : Find the densest region

Intuitive Description



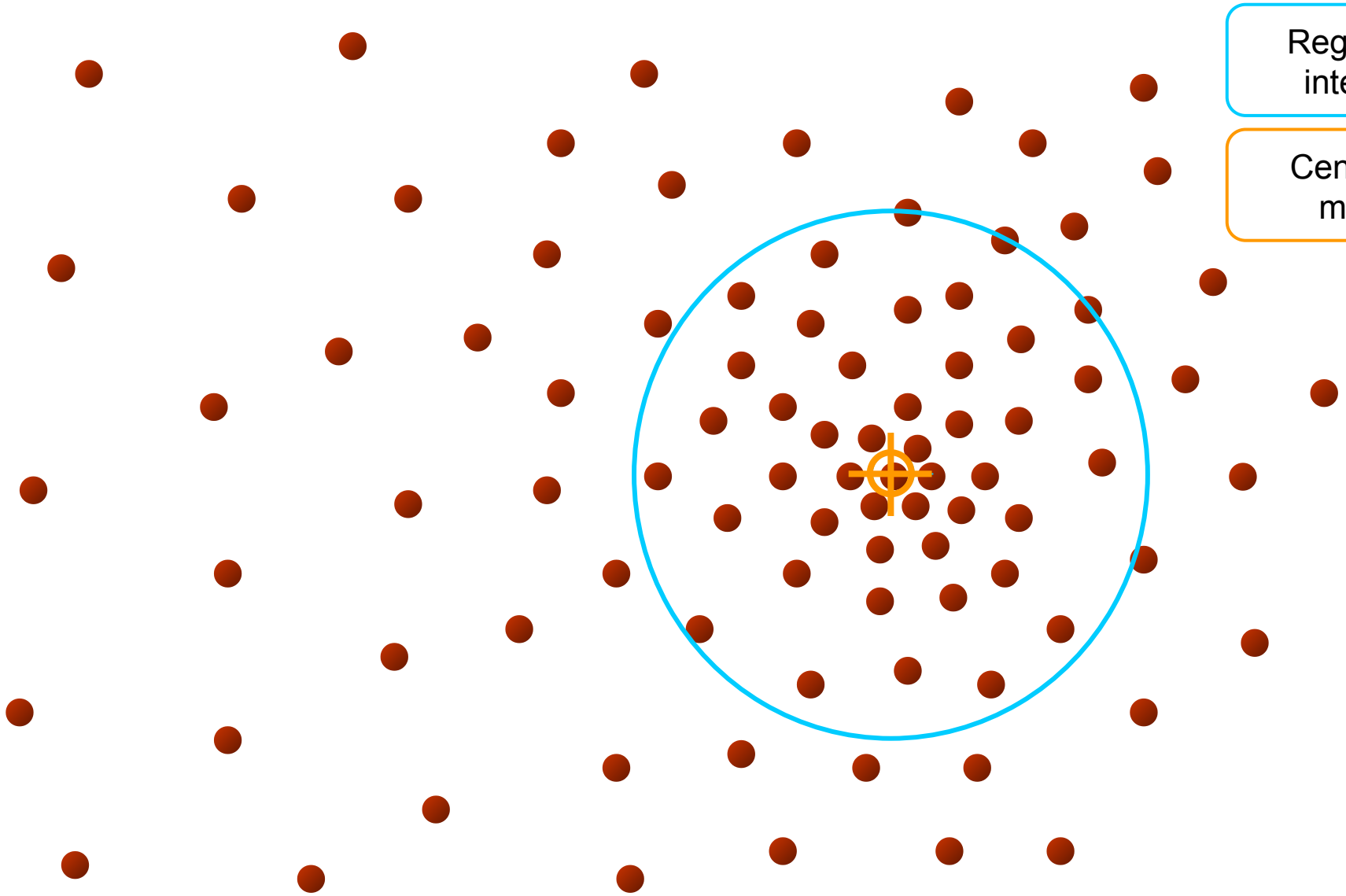
Objective : Find the densest region

Intuitive Description



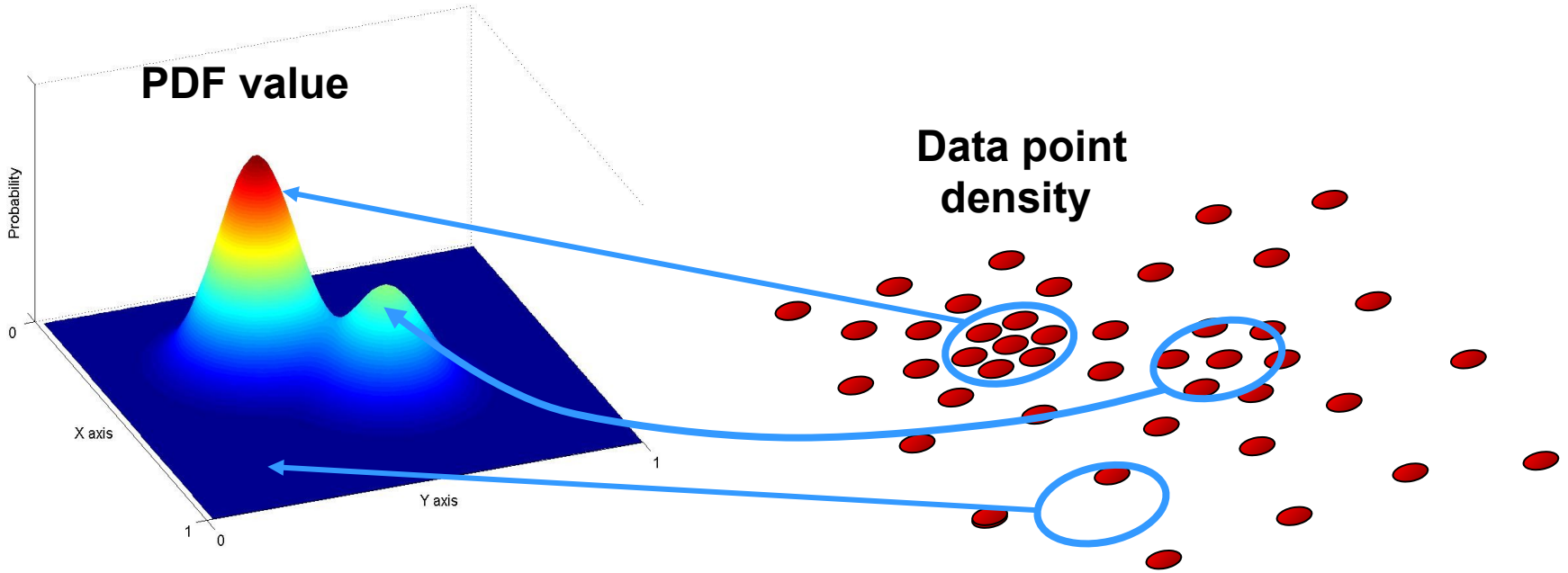
Objective : Find the densest region

Intuitive Description



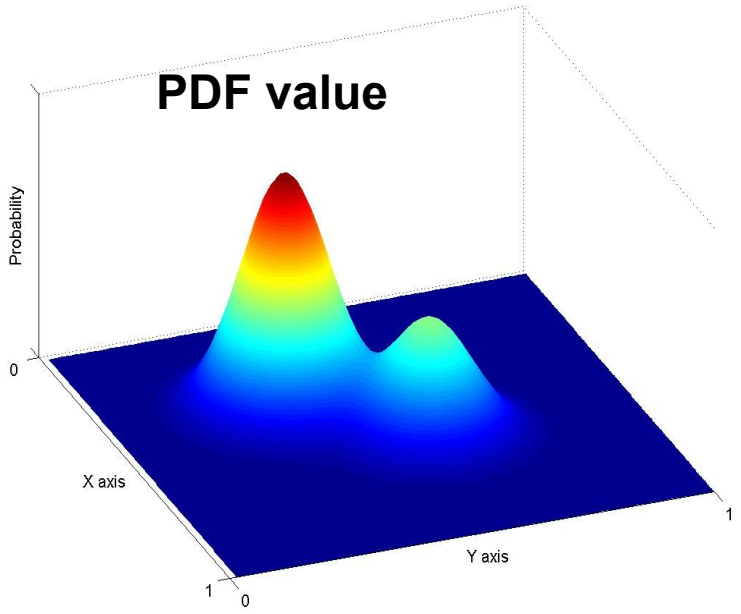
Objective : Find the densest region

Non-parametric Density Estimation

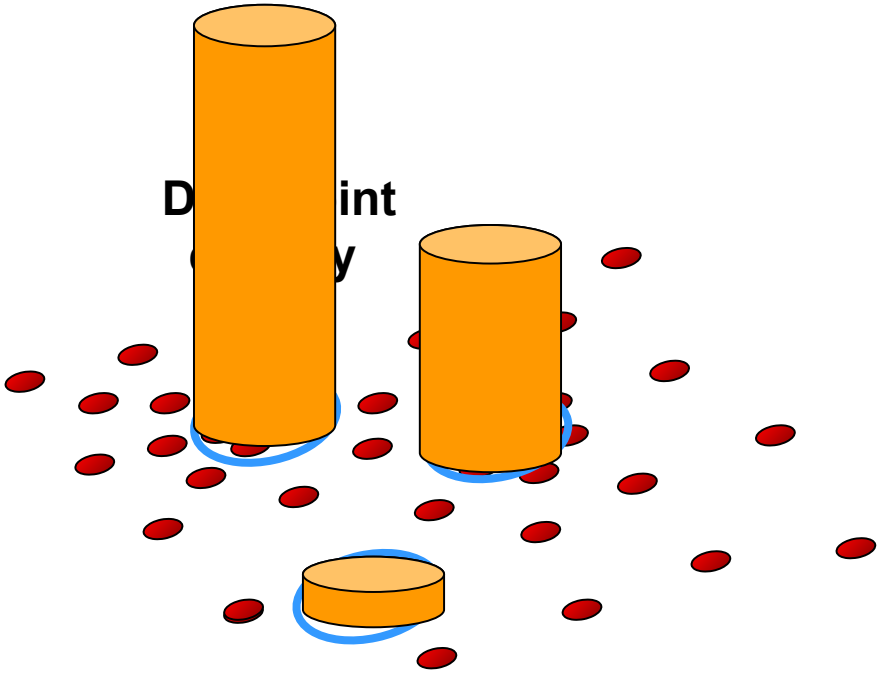


Assumed Underlying PDF **Data Samples**

Non-parametric Density Estimation

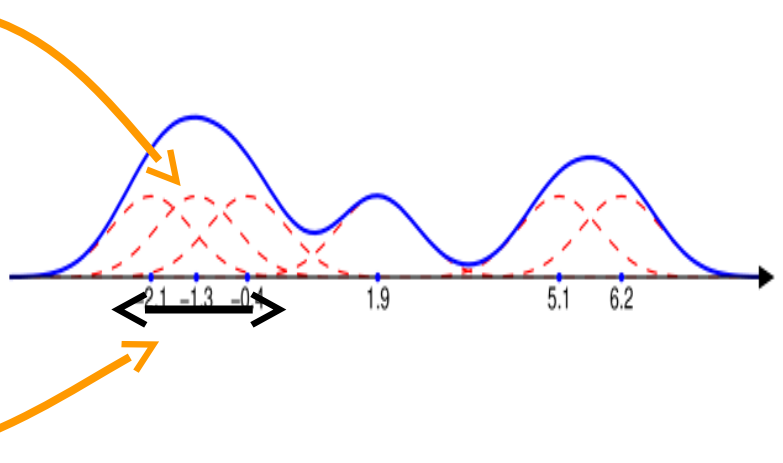


Assumed Underlying PDF



Data Samples

Parzen Windows



$$K_{\mathbf{H}}(\mathbf{x}) = |\mathbf{H}|^{-1/2} K(|\mathbf{H}|^{-1/2}\mathbf{x})$$

Kernel Properties

1. Bounded
2. Compact support
3. Normalized
4. Symmetric
5. Exponential decay
6. Uncorrelated

$$\int_{R^d} K(\mathbf{x}) d\mathbf{x} = 1$$

$$\lim_{||\mathbf{x}||\to\infty} ||\mathbf{x}||^d K(\mathbf{x}) d\mathbf{x} = 0$$

Kernels and Bandwidths

- Kernel Types

$$K^P(\mathbf{x}) = \prod_{i=1}^{d} K_1(x_i)$$

(product of univariate kernels)

$$K^S(\mathbf{x}) = a_{k,d} K_1(||\mathbf{x}||)$$

(radially symmetric kernel)

$$K(\mathbf{x}) = c_{k,d} k(||\mathbf{x}||^2)$$

- Bandwidth Parameter

$$\mathbf{H} = h^2 \mathbf{I}$$

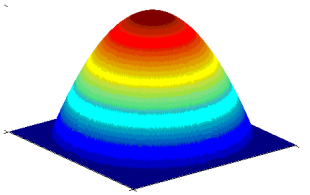
$$\hat{f}(\mathbf{x}) = \frac{c_{k,d}}{n h^d} \sum_{i=1}^{n} k\left(\left\| \frac{\mathbf{x} - \mathbf{x}_i}{h} \right\|^2 \right)$$

Various Kernels

Epanechnikov

$$k_E(x) = \begin{cases} 1-x & 0 \geq x \geq 1 \\ 0 & x > 1 \end{cases}$$

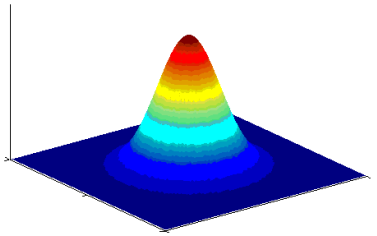
$$\Rightarrow K_E(\mathbf{x}) = \begin{cases} \frac{1}{2}c_d^{-1}(d+2)(1-\|\mathbf{x}\|^2) & \|\mathbf{x}\| \geq 1 \\ 0 & \|\mathbf{x}\| > 1 \end{cases}$$



Normal

$$k_N(x) = e^{-\frac{1}{2}x}$$

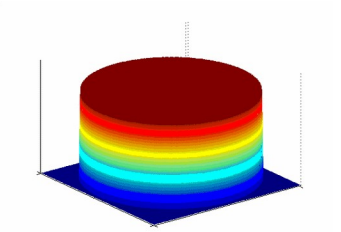
$$\Rightarrow K_N(\mathbf{x}) = (2\pi)^{-d/2} e^{-\frac{1}{2}\|\mathbf{x}\|^2}$$



Uniform

$$k_U(x) = \begin{cases} 1 & 0 \geq x \geq 1 \\ 0 & x > 1 \end{cases}$$

$$\Rightarrow K_U(\mathbf{x}) = \begin{cases} c_d & \|\mathbf{x}\| \leq 1 \\ 0 & \|\mathbf{x}\| > 1 \end{cases}$$



Density Gradient Estimation

$$\nabla \hat{f}_{h,K}(\mathbf{x}) = \frac{2c_{k,d}}{nh^{d+2}} \sum_{i=1}^{n} (\mathbf{x} - \mathbf{x}_i) k'\left(\left\| \frac{\mathbf{x}-\mathbf{x}_i}{h} \right\|^2 \right)$$

Modes of the probability density

$$\begin{aligned} g(x) &= -k'(x) \\ G(\mathbf{x}) &= c_{g,d} g(\|\mathbf{x}\|^2) \end{aligned}$$

Epanechnikov → Uniform

Normal → Normal

$$\begin{aligned} & \nabla \hat{f}_{h,K}(\mathbf{x}) \\ = & \frac{2c_{k,d}}{nh^{d+2}} \sum_{i=1}^{n} (\mathbf{x}_i - \mathbf{x}) g\left(\left\| \frac{\mathbf{x} - \mathbf{x}_i}{h} \right\|^2 \right) \\ = & \frac{2c_{k,d}}{nh^{d+2}} \left[\sum_{i=1}^{n} g\left(\left\| \frac{\mathbf{x} - \mathbf{x}_i}{h} \right\|^2 \right) \right] \left[\frac{\sum_{i=1}^{n} \mathbf{x}_i g\left(\left\| \frac{\mathbf{x}-\mathbf{x}_i}{h} \right\|^2 \right)}{g\left(\left\| \frac{\mathbf{x}-\mathbf{x}_i}{h} \right\|^2 \right)} - \mathbf{x} \right] \end{aligned}$$

Mean Shift

$$\nabla \hat{f}_{h,K}(\mathbf{x})$$

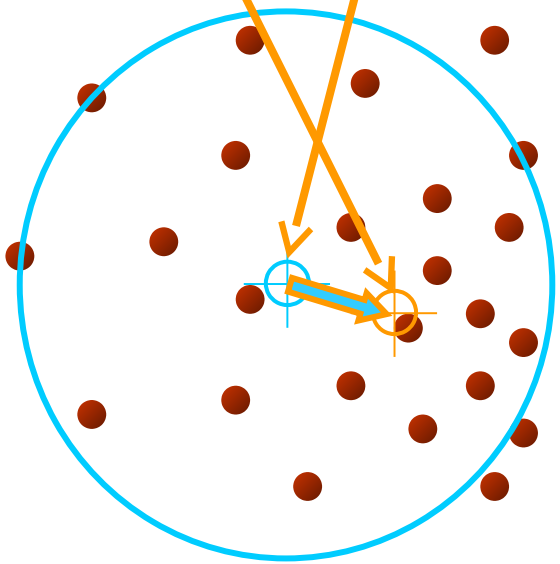
$$= \frac{2c_{k,d}}{nh^{d+2}} \underbrace{\left[\sum_{i=1}^{n} g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)\right]}_{\text{KDE}} \underbrace{\left[\frac{\sum_{i=1}^{n} \mathbf{x}_i g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)}{g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)} - \mathbf{x}\right]}_{\text{Mean Shift}}$$

Mean Shift Algorithm

- compute mean shift vector

$$\mathbf{m}_{h,G}(\mathbf{x}) = \left[\frac{\sum_{i=1}^{n} \mathbf{x}_i g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)}{g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)} - \mathbf{x}\right]$$

- translate kernel (window) by mean shift vector



Mean Shift

$$\nabla \hat{f}_{h,K}(\mathbf{x}) = \hat{f}_{h,G}(\mathbf{x}) \frac{2c_{k,d}}{h^2 c_{g,d}} \mathbf{m}_{h,G}(\mathbf{x})$$

$$\Rightarrow \mathbf{m}_{h,G}(\mathbf{x}) = \frac{1}{2} h^2 c \frac{\nabla \hat{f}_{h,K}(\mathbf{x})}{\hat{f}_{h,G}(\mathbf{x})}$$

- Mean Shift is proportional to the *normalized* density gradient estimate obtained with kernel K
- The normalization is by the density estimate computed with kernel G

Properties of Mean Shift

- Guaranteed convergence
  - Gradient Ascent algorithms are guaranteed to converge only for infinitesimal steps.
  - The normalization of the mean shift vector ensures that it converges.
  - Large magnitude in low-density regions, refined steps near local maxima → Adaptive Gradient Ascent.

- Mode Detection
  - Let $\{\mathbf{y}_j\}_{j=1,2,\cdots}$ denote the sequence of kernel locations.
  - At convergence $\mathbf{m}_{h,G}(\mathbf{y}_c) = \mathbf{y}_c - \mathbf{y}_c = 0 \Rightarrow \nabla \hat{f}_{h,K}(\mathbf{y}_c) = 0$
  - Once $\mathbf{y}_j$ gets sufficiently close to a mode of $\hat{f}_{h,K}$ it will converge to the mode.
  - The set of all locations that converge to the same mode define the *basin of attraction* of that mode.

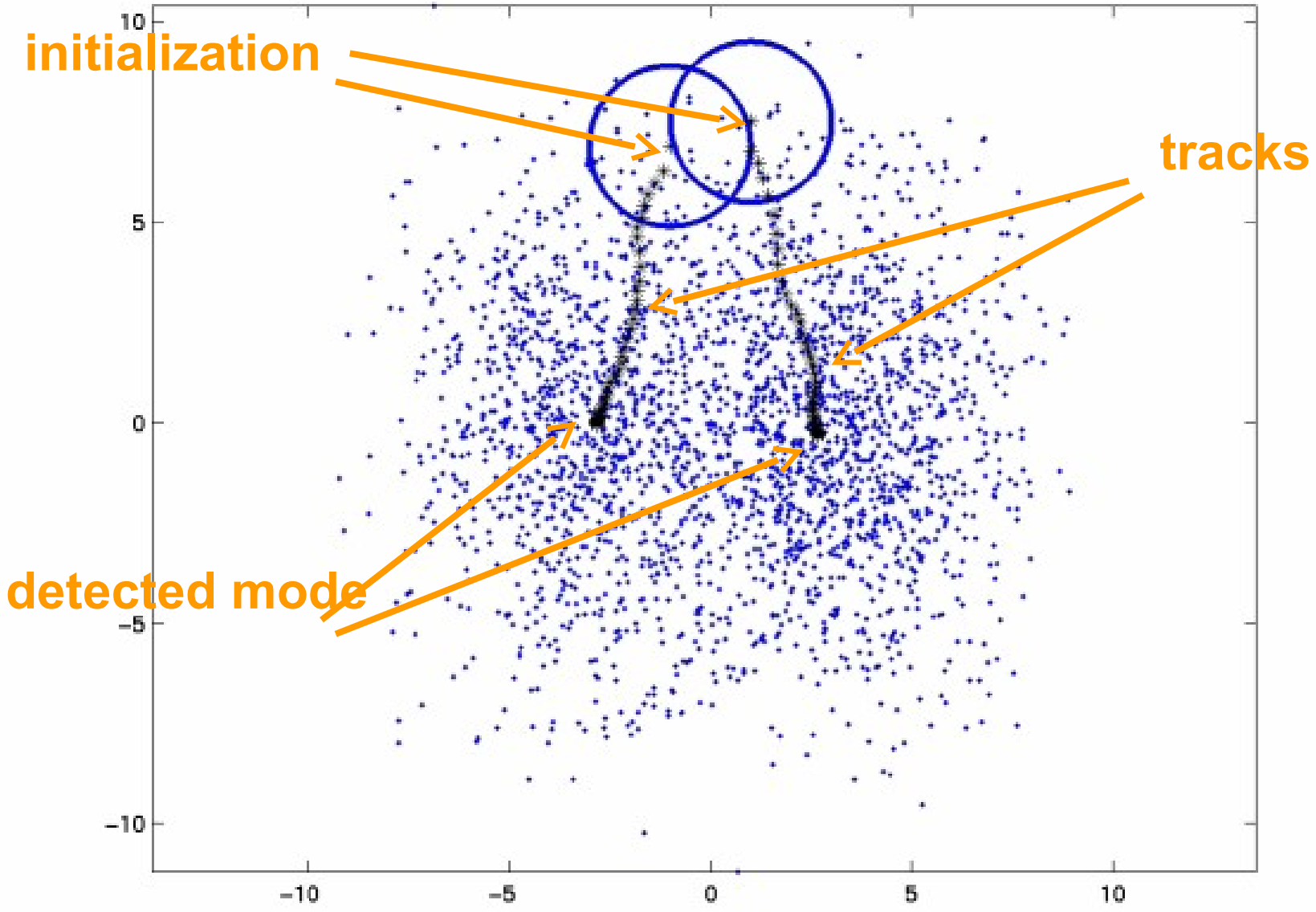
Properties of Mean Shift

- Smooth Trajectory
  - The angle between two consecutive mean shift vectors computed using the normal kernel is always less that 90°
  - In practice the convergence of mean shift using the normal kernel is very slow and typically the uniform kernel is used.

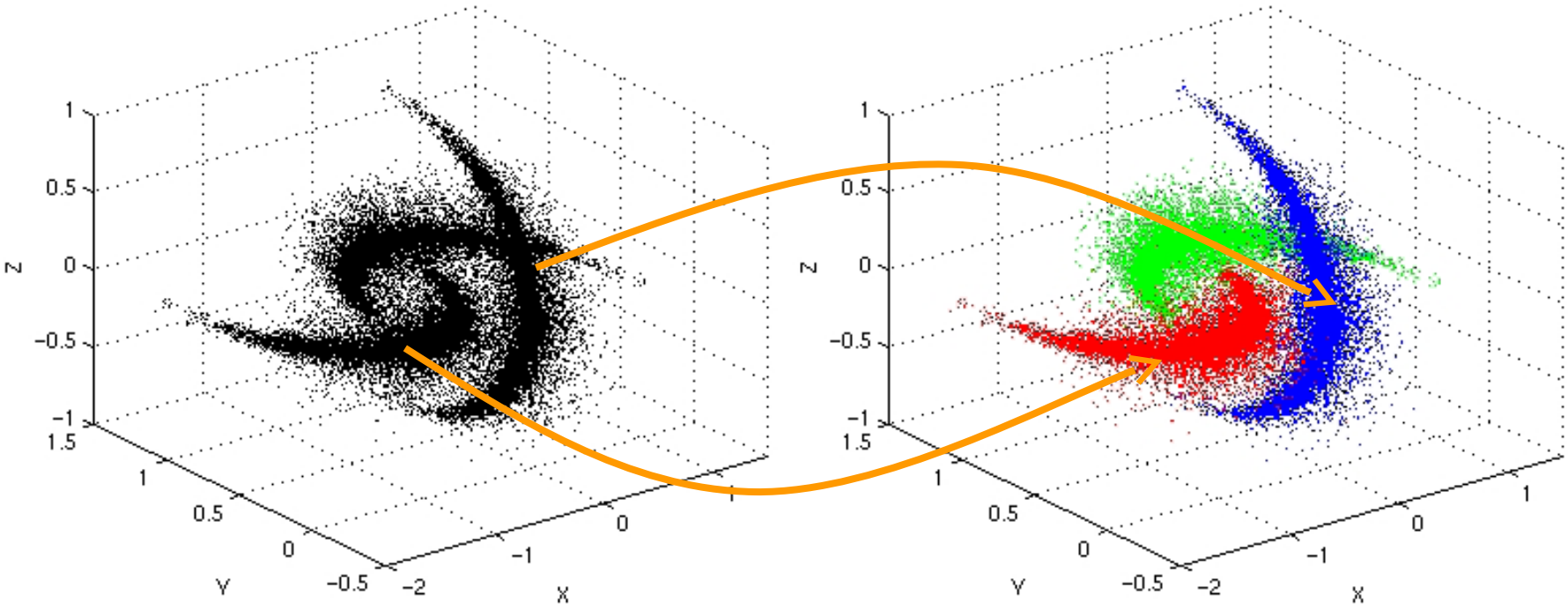
Mode detection using Mean Shift

- Run Mean Shift to find the stationary points
  - To detect multiple modes, run in parallel starting with initializations covering the entire feature space.
- Prune the stationary points by retaining local maxima
  - Merge modes at a distance of less than the bandwidth.
- Clustering from the modes
  - The basin of attraction of each mode delineates a cluster of arbitrary shape.

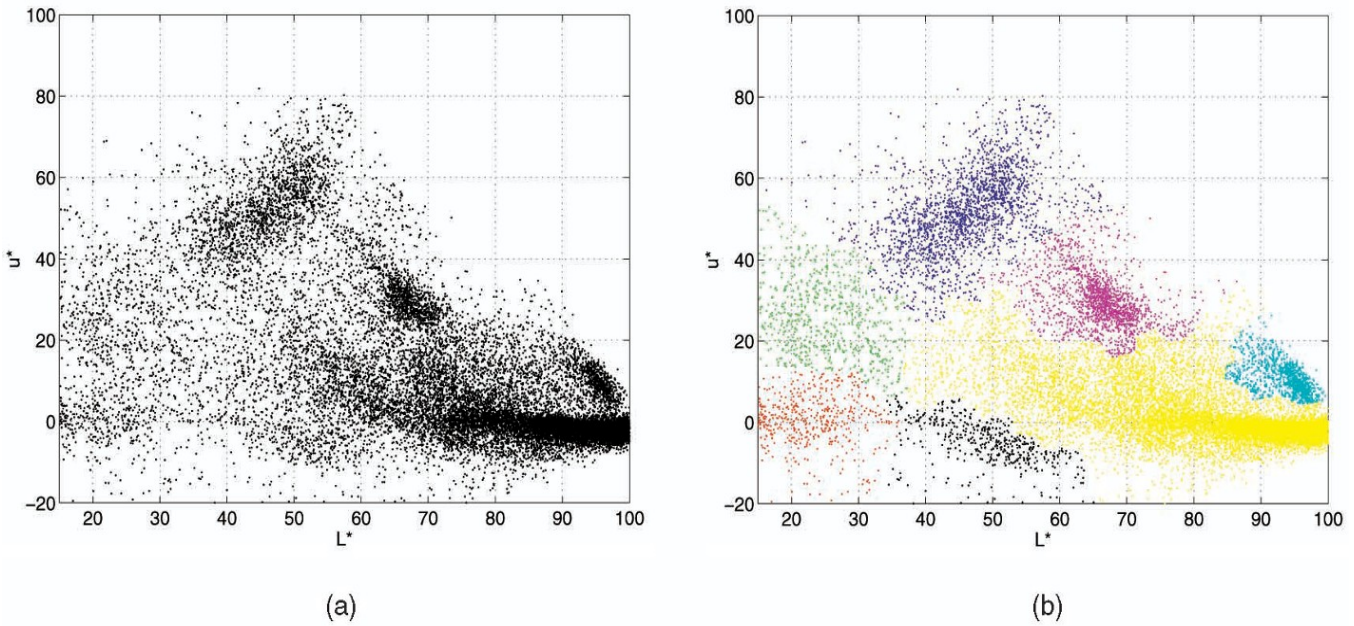
Mode Finding on Real Data

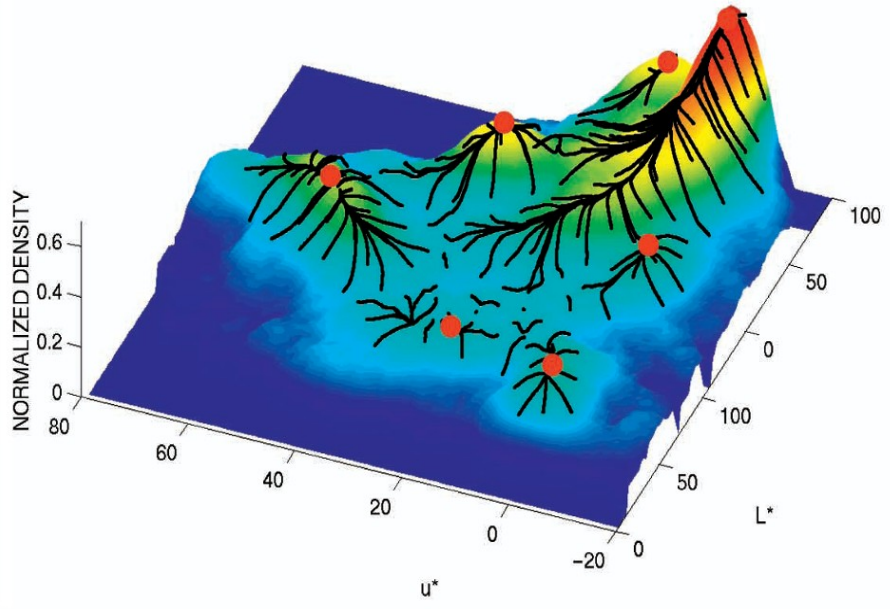


Mean Shift Clustering



Clustering on Real Data





(c)

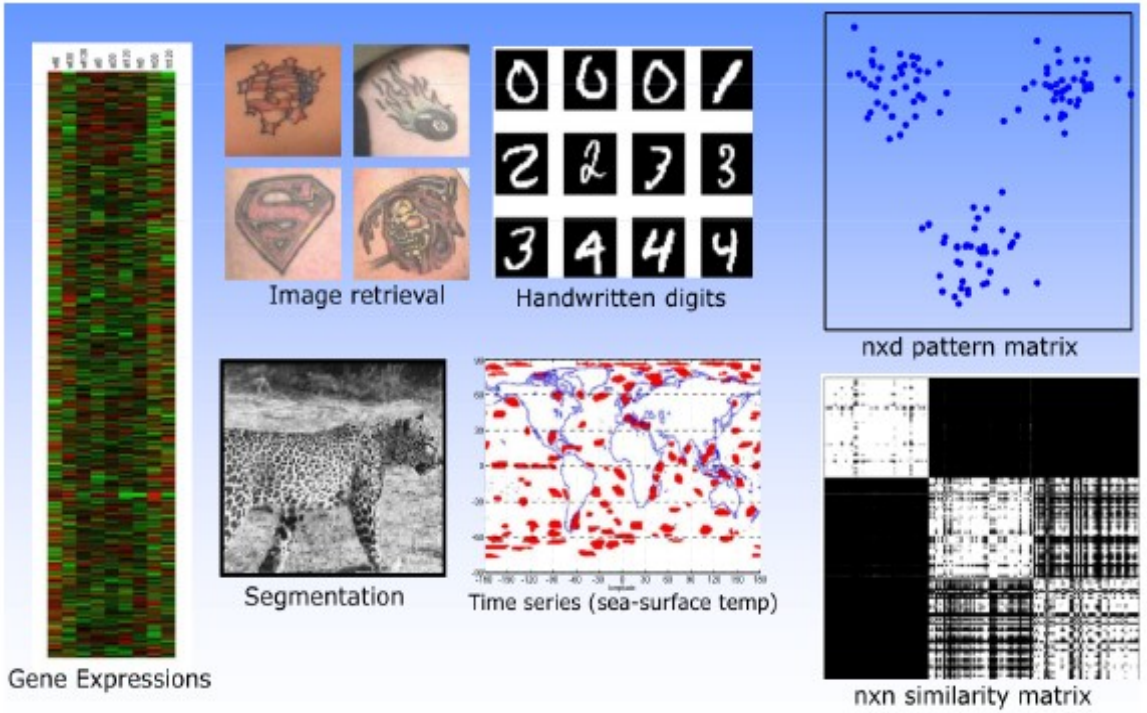
Mean Shift Segmentation

Notes on implementation

- Tracing the tracks for each point can be too slow for image segmentation.
- There are two common heuristics used to speedup the algorithm:
  1) **Basin of attraction:** Upon finding a peak, associate each data point that is at a distance r from the peak with the cluster dened by that peak.
  2) Points that are within a distance of r/c of the search path are associated with the converged peak, where c is some constant value. c = 4 is a common value of image segmetnation.

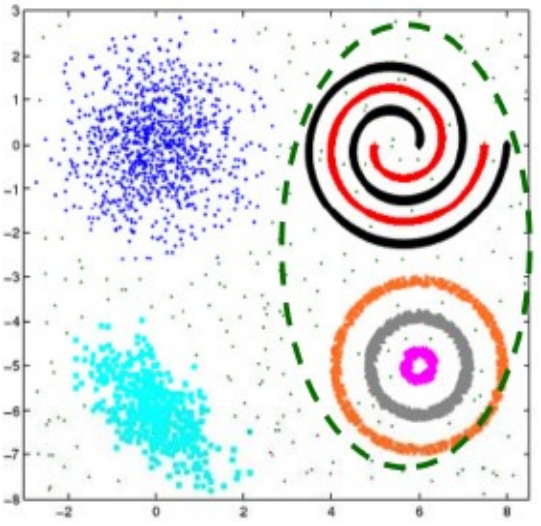
- Gaussian mixture models and the K-means algorithm make use of the Euclidean distance between points.
- Why should the points be compared in this manner?
- In many cases the vector representation of objects to be clustered is derived, i.e., comes from some feature transformation.
- It is not at all clear that the Euclidean distance is an appropriate way of comparing the resulting feature vectors.
- Features might not give raise to vectors
  - Different cardinality (parts and relations)
  - Mixed continuous and
    Categorical data

- There's **no universal representation**;
  they're domain dependent



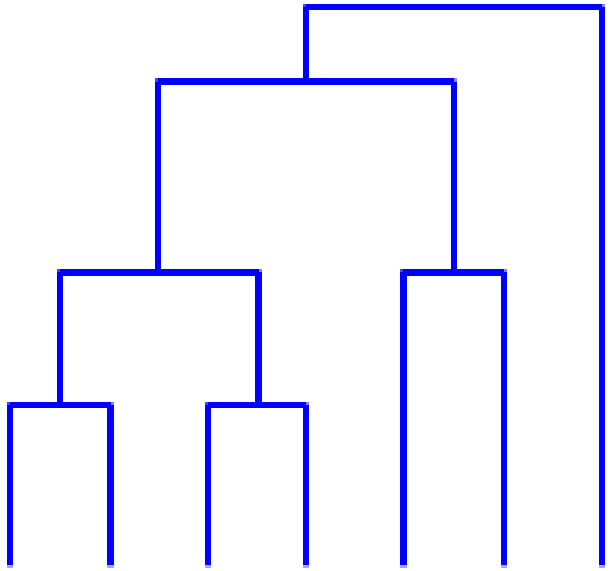
- **Distance Metrics**
  - Euclidean distance
  - Hamming distance (number of mismatches between two strings)
  - Travel distance along a manifold (e.g. for geographic points)
  - Tempo / rhythm similarity (for songs)
  - Shared keywords (for web pages), or shared in-links
- **Scoring functions**
  - Minimize: Summed distances between all pairs of objects in the same cluster. (Also known as ”within-cluster scatter.”)
  - Minimize: Maximum distance between any two objects in the same cluster. (Can be hard to optimize.)
  - Maximize: Minimum distance between any two objects in different clusters.

- **Compact Clusters:**
  - Within-cluster distance < between-cluster connectivity
- **Connected Clusters:**
  - Within-cluster connectivity > between-cluster connectivity



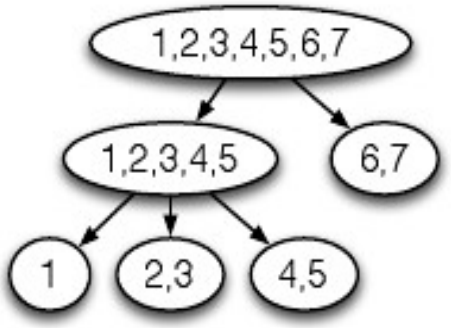
Hierarchical clustering

- Organizes data instances into trees.
- For visualization, exploratory data analysis.
- **Agglomerative methods:** build the tree bottom-up, successively grouping together the clusters deemed most similar.
- **Divisive methods:** build the tree top-down, recursively partitioning the data.
-



- Given instances D = {x1, . . . , xm}. A hierarchical clustering is a set of subsets (clusters) of D, C = {C1, . . . , CK }, where
  - Every element in D is in at least one set of C (the root)
  - The Cj can be assigned to the nodes of a tree such that the cluster at any node is precisely the union of the clusters at the node's children (if any).

- Suppose D = {1, 2, 3, 4, 5, 6, 7}. A hierarchical clustering is
  C ={{1}, {2, 3}, {4, 5}, {1, 2, 3, 4, 5}, {6, 7}, {1, 2, 3, 4, 5, 6, 7}}.



- In this example:
  - Leaves of the tree need not correspond to single instances.
  - The branching factor of the tree is not limited.
- However, most hierarchical clustering algorithms produce binary trees, and take single instances as the smallest clusters.

Aglomerative clustering

- Input: Pairwise distances d(x, x') between a set of data objects $\{x_i\}$.
- Output: A hierarchical clustering
- Algorithm:
  1. Assign each instance as its own cluster on a working list W .
  2. Repeat
     a) Find the two clusters in W that are most "similar".
     b) Remove them from W .
     c) Add their union to W .

     until W contains a single cluster with all the data objects.
  3. Return all clusters appearing in W at any stage of the algorithm.

- How many clusters are generated by the agglomerative clustering algorithm?
- Answer: 2m − 1, where m is the number of data objects.
  - A binary tree with m leaves has m − 1 internal nodes, thus 2m − 1 nodes total.
- More explicitly:
  - The working list W starts with m singleton clusters
  - Each iteration removes two clusters from W and adds one new one
  - The algorithm stops when W has one cluster, which is after m − 1 iterations

How do we measure dissimilarity between clusters?

- Distance between nearest objects ("Single-linkage" agglomerative
- clustering, or "nearest neighbor"):

$$\min_{\mathbf{x}\in C,\mathbf{x}'\in C'} d(\mathbf{x},\mathbf{x}')$$

- Distance between farthest objects ("Complete-linkage" agglomerative
- clustering, or "furthest neighbor"):

$$\max_{\mathbf{x}\in C,\mathbf{x}'\in C'} d(\mathbf{x},\mathbf{x}')$$

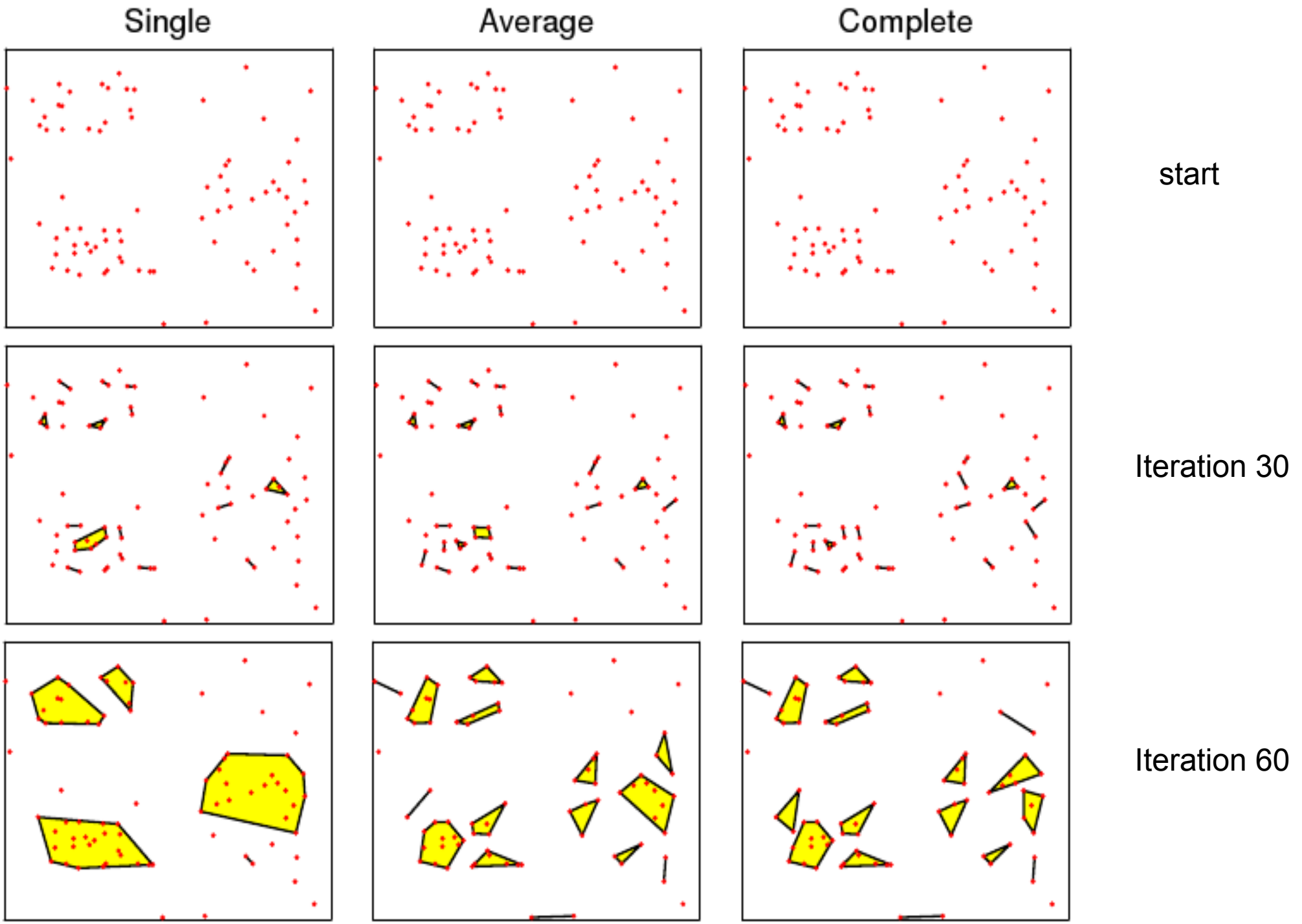
- Average distance between objects ("Group-average" agglomerative
- Clustering):

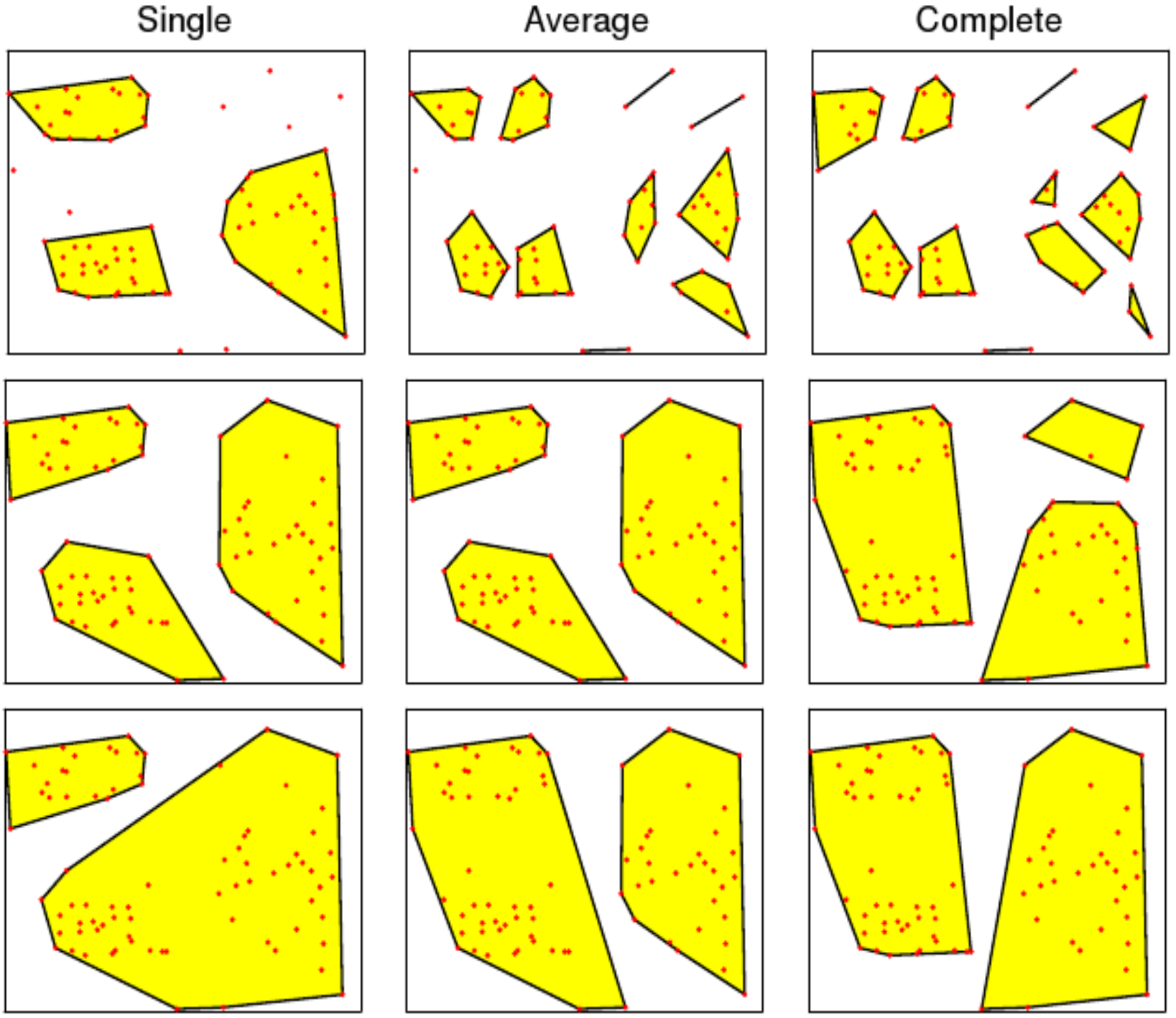
$$\frac{1}{|C||C'|}\sum_{\mathbf{x}\in C,\mathbf{x}'\in C'} d(\mathbf{x},\mathbf{x}')$$

Intuition

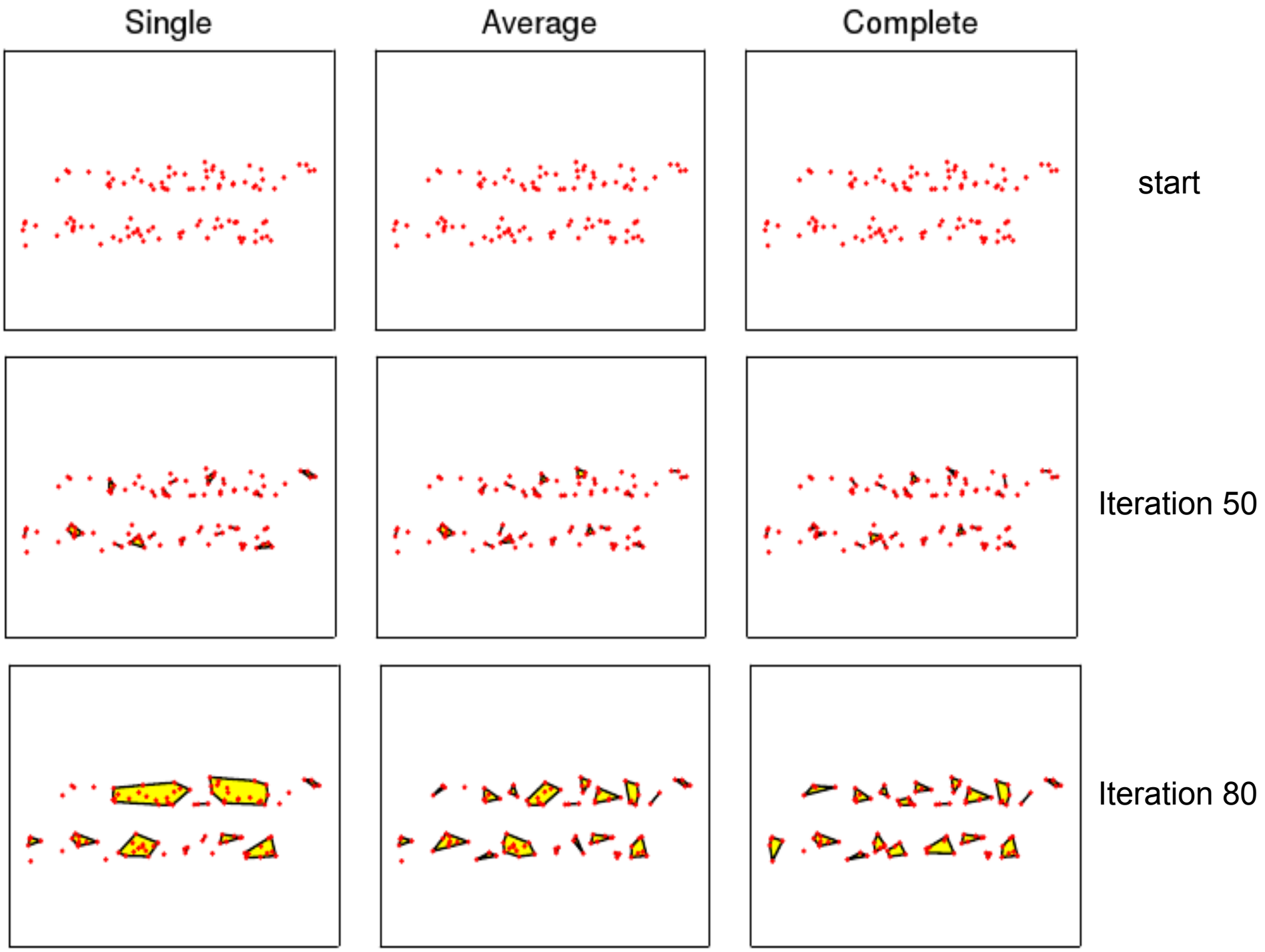
- Single-linkage
  - Favors spatially-extended / filamentous clusters
  - Often leaves singleton clusters until near the end
- Complete-linkage favors compact clusters
- Average-linkage is somewhere in between

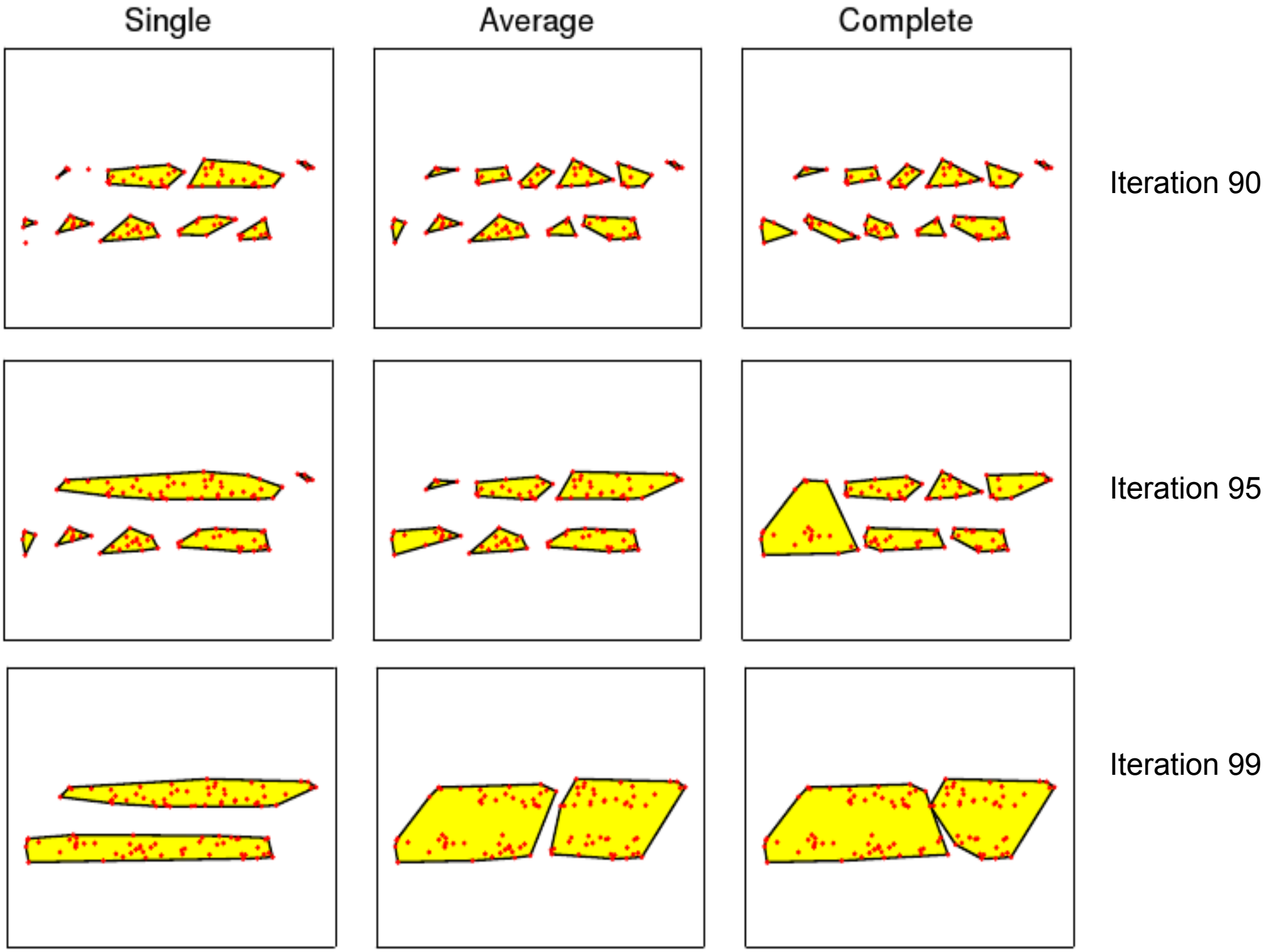
Example 1




Single
Average
Complete
Iteration 70
Iteration 78
Iteration 79

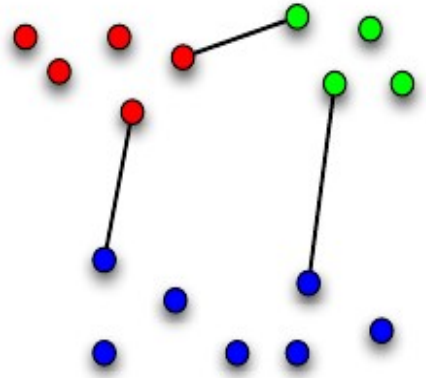
Example 2




Single
Average
Complete
Iteration 90
Iteration 95
Iteration 99

Monotonicity

- Let A, B, C be clusters.
- Let d be one of the dissimilarity measures: single-linkage, average linkage or complete linkage
- If d(A, B) ≤ d(A, C) and d(A, B) ≤ d(B, C), then d(A, B) ≤ d(A∪B, C)
-
-
-
-
-
- Proof (single link)

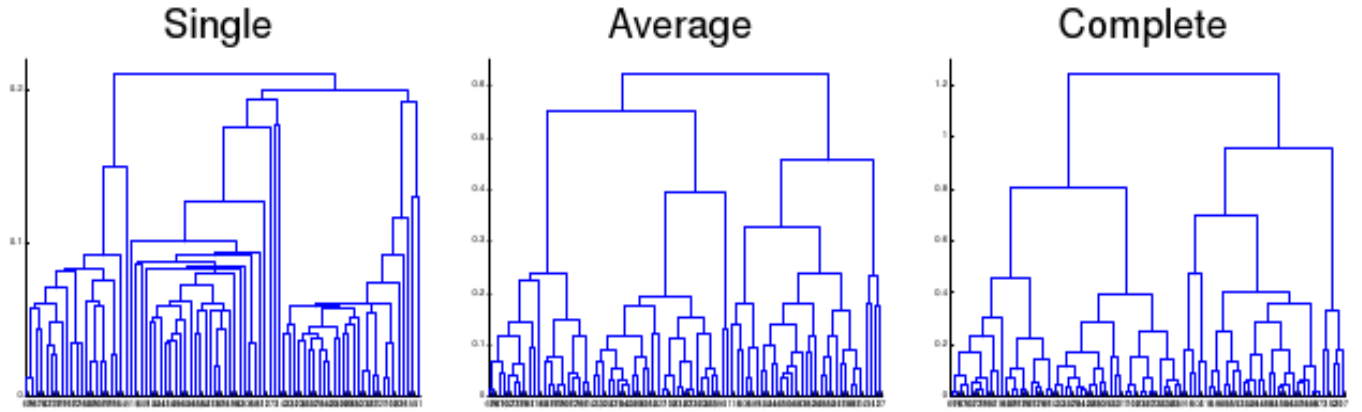


Suppose that that d(A, B) ≤ d(A, C) and d(A, B) ≤ d(B, C), then

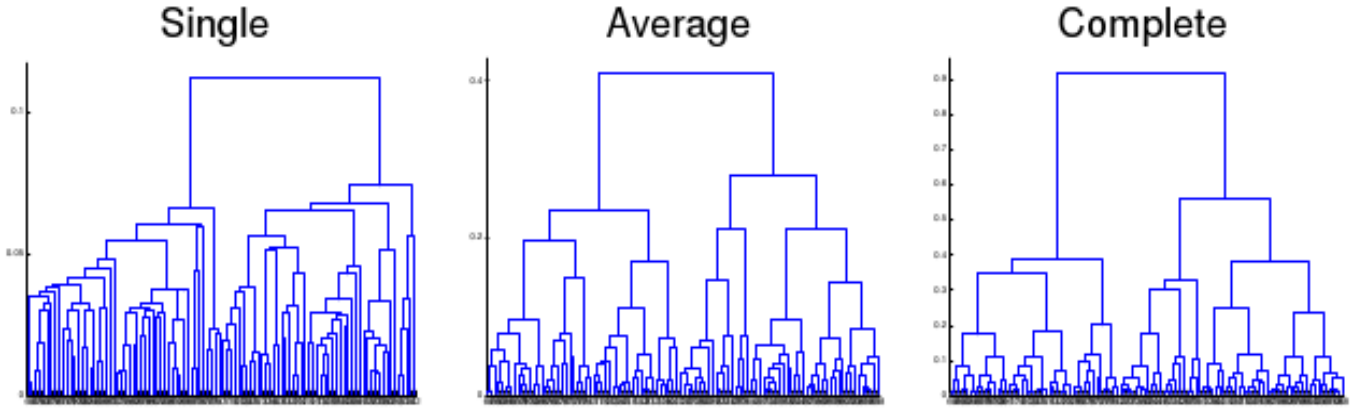
$$
\begin{aligned}
d(A \cup B, C) &= \min_{x \in A \cup B, x' \in C} d(x, x') \\
&= \min \left(\min_{x_a \in A, x' \in C} d(x_a, x'), \min_{x_b \in B, x' \in C} d(x_a, x') \right) \\
&= \min \left(d(A, C), d(B, C) \right) \\
&\geq \min \left(d(A, B), d(A, B) \right) \\
&= d(A, B)
\end{aligned}
$$

Dendrograms

- The monotonicity property implies that every time agglomerative clustering merges two clusters, the dissimilarity of those clusters is ≥ the dissimilarity of all previous merges.
- Dendrograms (trees depicting hierarchical clusterings) are often drawn so that the height of a node corresponds to the dissimilarity of the merged clusters.
- We can form a flat clustering by cutting the tree at any height.
- Jumps in the height of the dendrogram can suggest natural cutoffs.



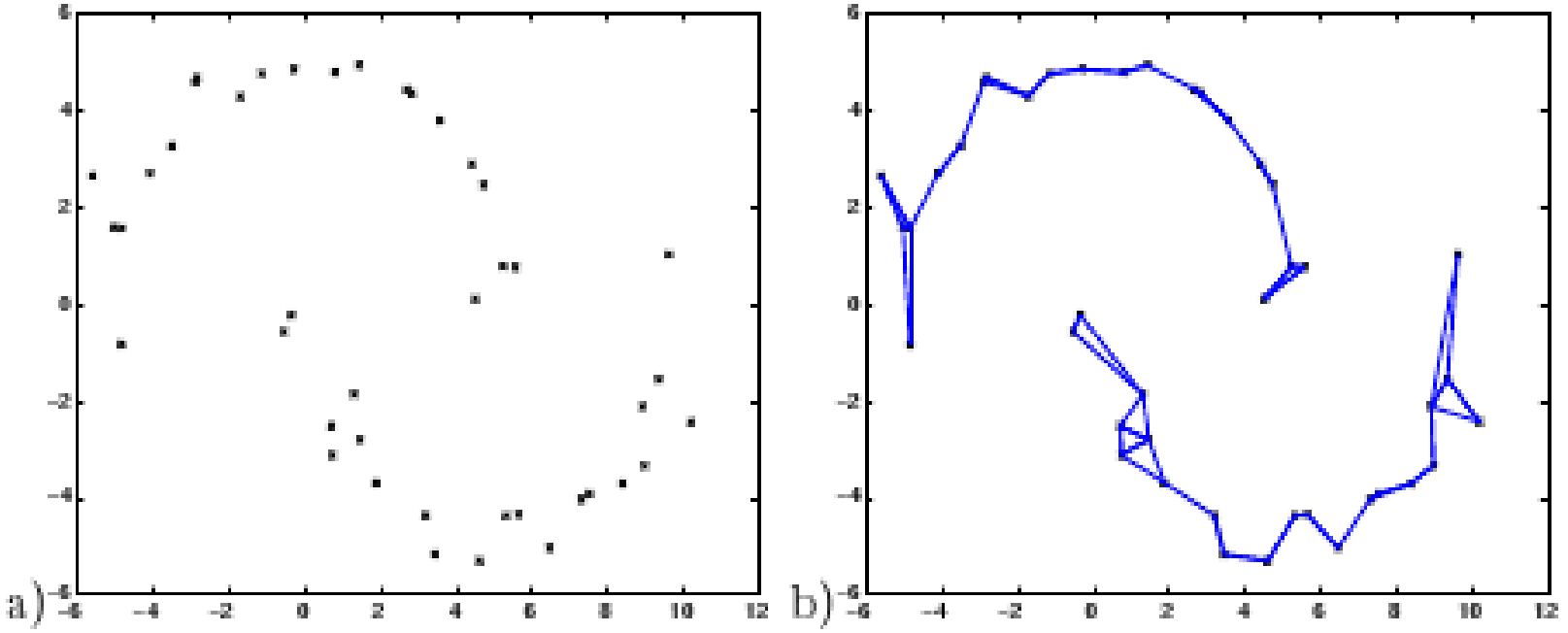
Example 1



Example 2

Spectral (Graph-based) clustering

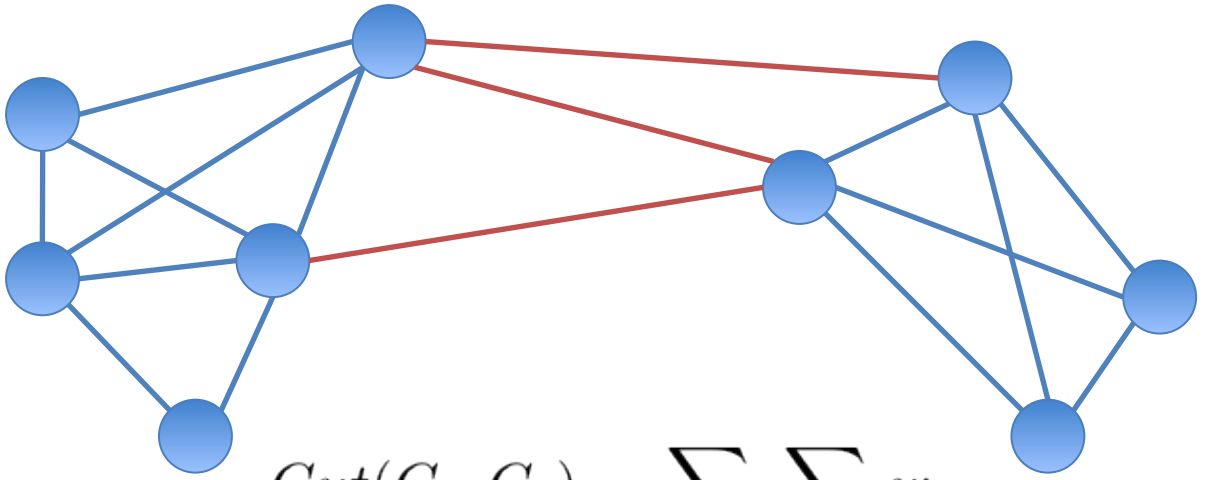
- Spectral clustering refers to a class of clustering methods that approximate the problem of partitioning nodes in a weighted graph.
- The weighted graph represents a similarity matrix between the objects associated with the nodes in the graph.
- A large positive weight connecting any two nodes (high similarity) biases the clustering algorithm to place the nodes in the same cluster.
- The graph representation is relational in the sense that it only holds information about the comparison of objects associated with the nodes.



- A relational representation can be advantageous even in cases where a vector space representation is readily available.

Graphs Cuts

- The **cut** between two subgraphs is calculated as follows



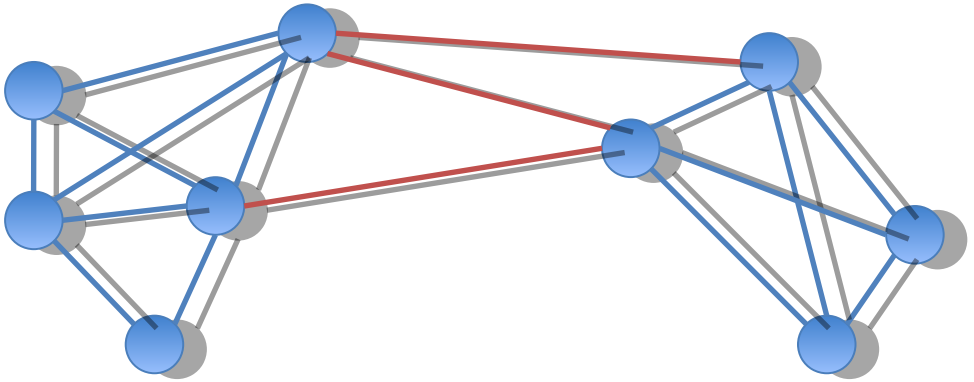
$$Cut(C_1, C_2) = \sum_{i \in C_1} \sum_{j \in C_2} w_{ij}$$

- The minimum cut of a graph identifies an optimal partitioning of the data.

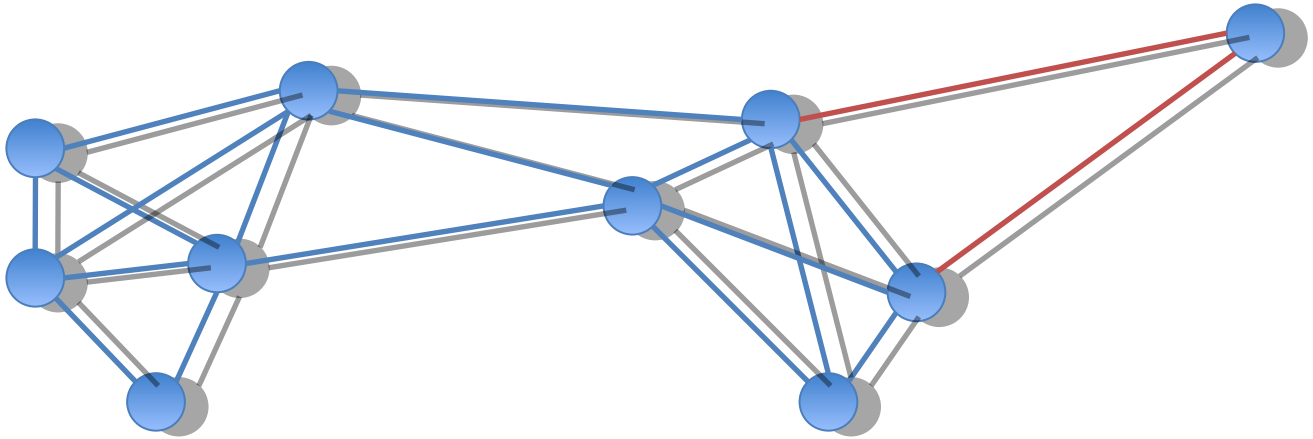
- Spectral Clustering
  - Recursively partition the data set
    - Identify the minimum cut
    - Remove edges
    - Repeat until k clusters are identified

- Minimum (bipartitional) cut

$$\min Cut(C_1, C_2) = \sum_{i \in C_1} \sum_{j \in C_2} w_{ij}$$



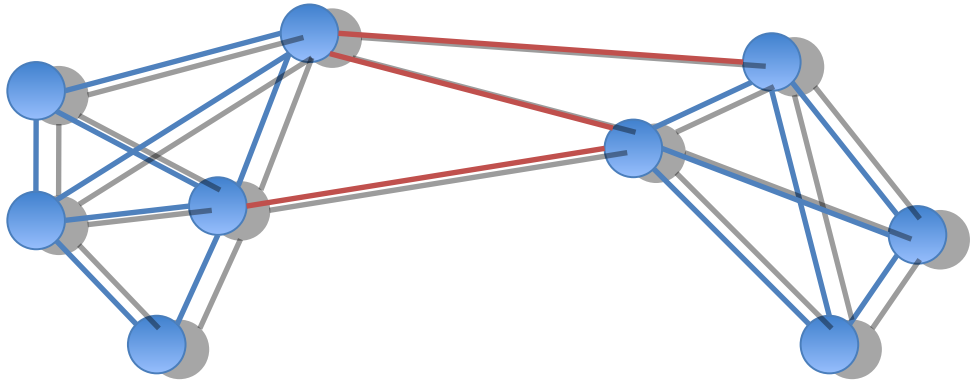
- Unnormalized cuts are attracted to outliers.

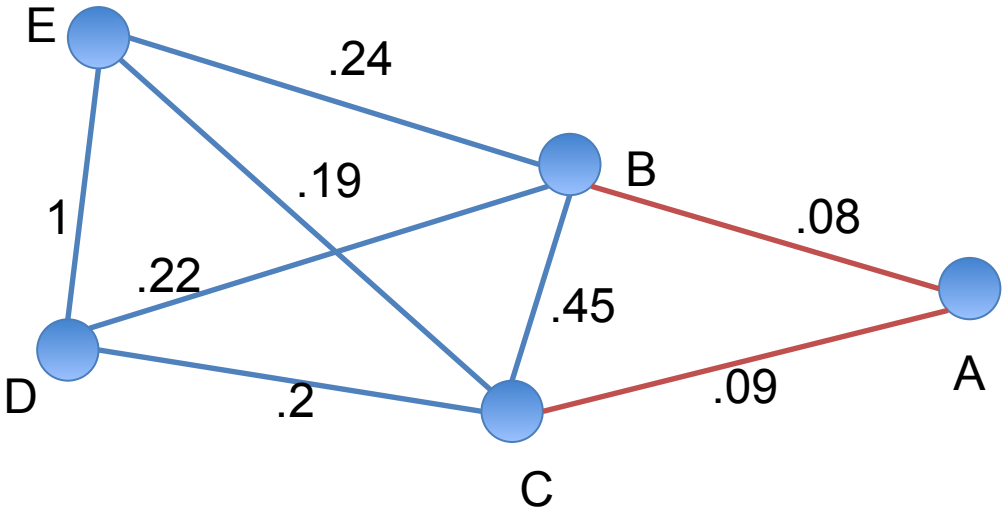


Normalized cut

$$\min \frac{Cut(C_1, C_2)}{Vol(C_1)} + \frac{Cut(C_1, C_2)}{Vol(C_2)} = \min \left(\frac{1}{Vol(C_1)} + \frac{1}{Vol(C_2)} \right) Cut(C_1, C_2)$$

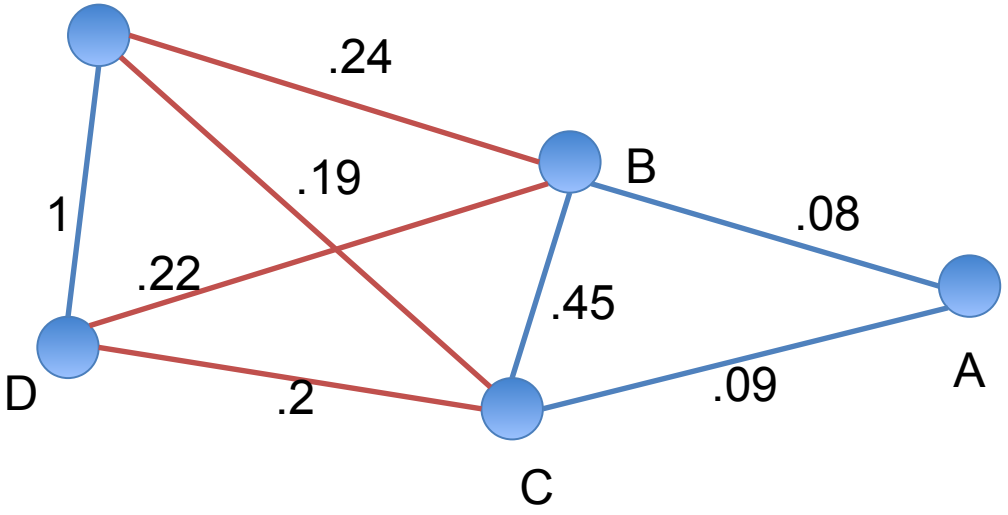
$$Vol(C) = \sum_{i \in C} D(x_i)$$





$$Cut(BCDE, A) = 0.17$$

$$NormCut(BCDE, A) = 1.067$$



$$Cut(ABC, DE) = 0.95$$

$$NormCut(ABC, DE) = 1.038$$

Problem

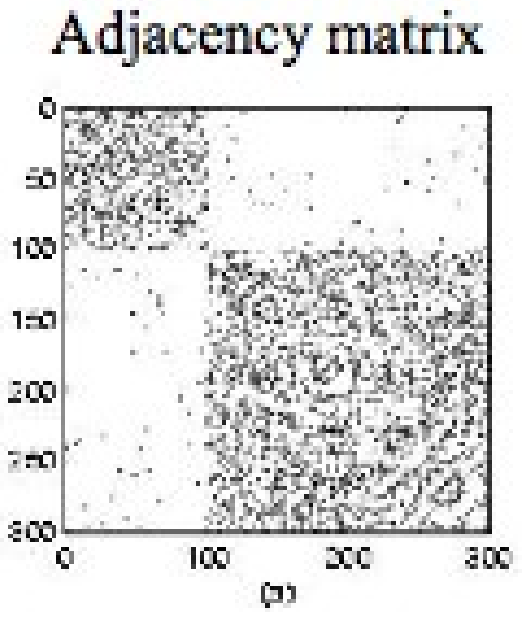
- Identifying a minimum normlaized cut is NP-hard.
- There are efficient approximations using linear algebra.
- Based on the Laplacian Matrix, or **graph Laplacian**

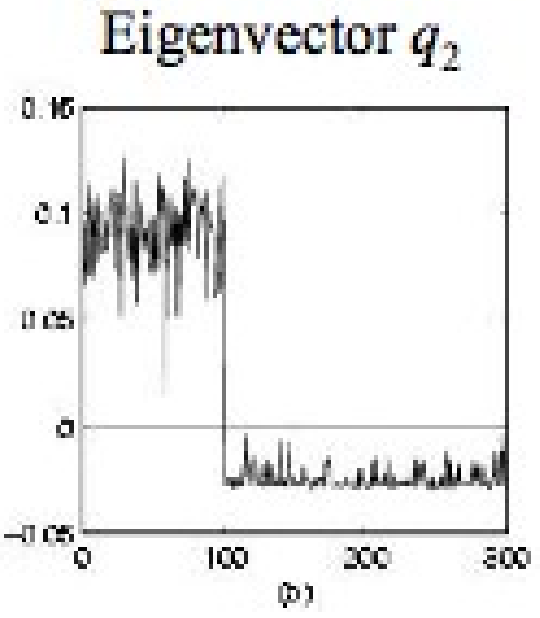
$$L = D - W$$

with D diagonal **degree** matrix

$$d_{ii} = \sum_j w_{ij}$$

- Sign of the components of the eigenvectors of the Laplacian matrix are related to the cuts





Why does this work?

- How does this eigenvector decomposition relate to cuts?
- Let $f_i \in \{-1, 1\}$ be a cluster assignment for node i

$$
\begin{aligned}
f^T L f &= f^T D f - f^T W f \\
&= \sum_i d_i f_i^2 - \sum_{ij} f_i f_j w_{ij} \\
&= \frac{1}{2}\left(\sum_i \left(\sum_j w_{ij}\right) f_i^2 - 2\sum_{ij} f_i f_j w_{ij} + \sum_j \left(\sum_i w_{ij}\right) f_j^2\right) \\
&= \frac{1}{2}\sum_{ij} w_{ij}(f_i - f_j)^2
\end{aligned}
$$

This quantity is:
0 if $f_i = f_j$
$4w_{ij}$ otherwise

- $f^T L f$ is the cut value
- Note that $||f|| = \sqrt{\sum_{i=1}^{n} f_i^2} = \sqrt{n}$
- Relax requirement $f_i \in \{-1, 1\}$ to $||f|| = \sqrt{n}$ and compute *f* as smallest non-constant eigenvector

Normalized cut

$$\min \frac{Cut(C_1, C_2)}{Vol(C_1)} + \frac{Cut(C_1, C_2)}{Vol(C_2)} = \min \left(\frac{1}{Vol(C_1)} + \frac{1}{Vol(C_2)} \right) Cut(C_1, C_2)$$

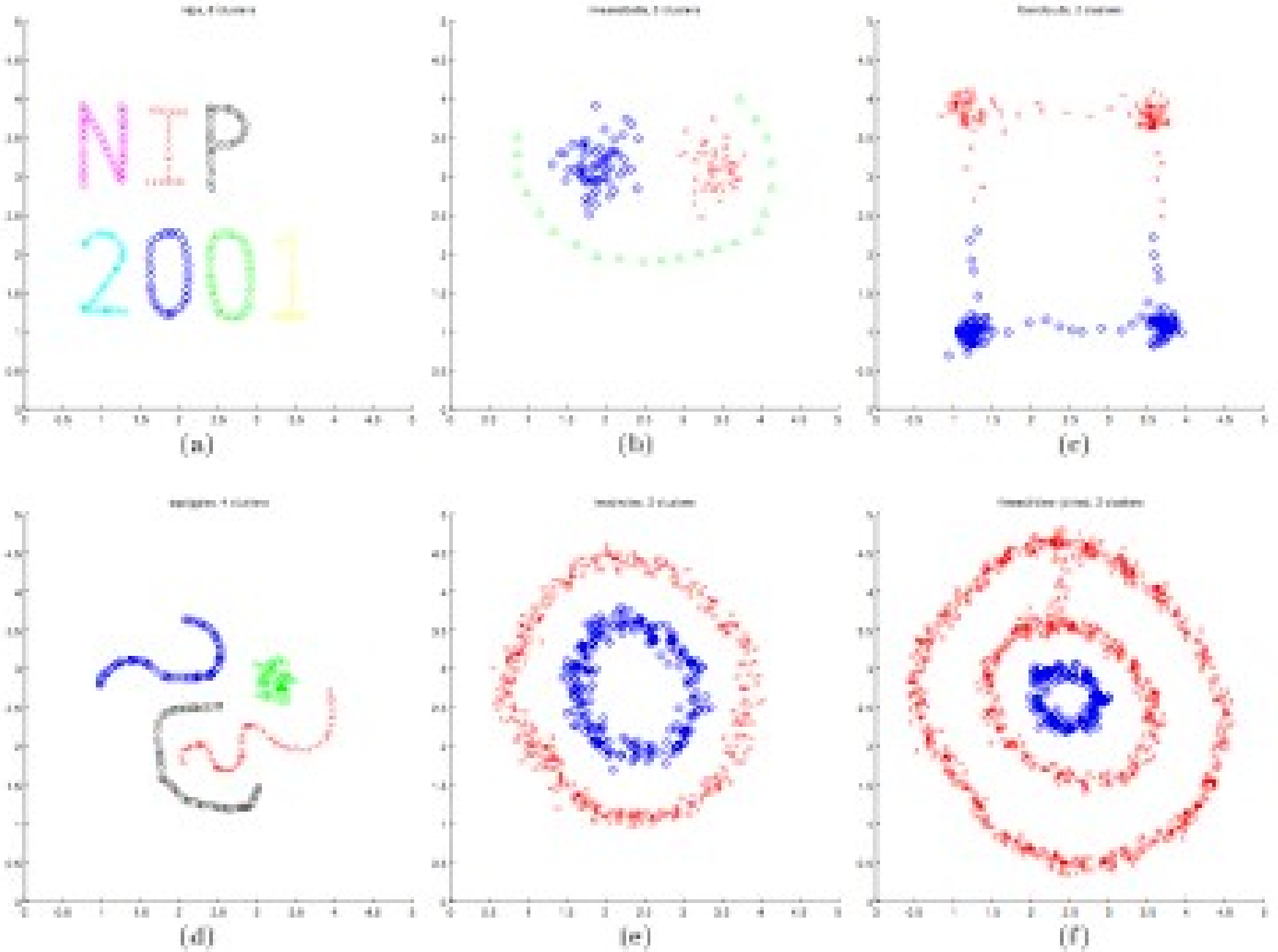
$$NCut(A, B) = \frac{y^T (D - W) y}{y^T D y}$$

- After relaxation $\min_y y^T (D - W) y \text{ subject to } y^T D y = 1$

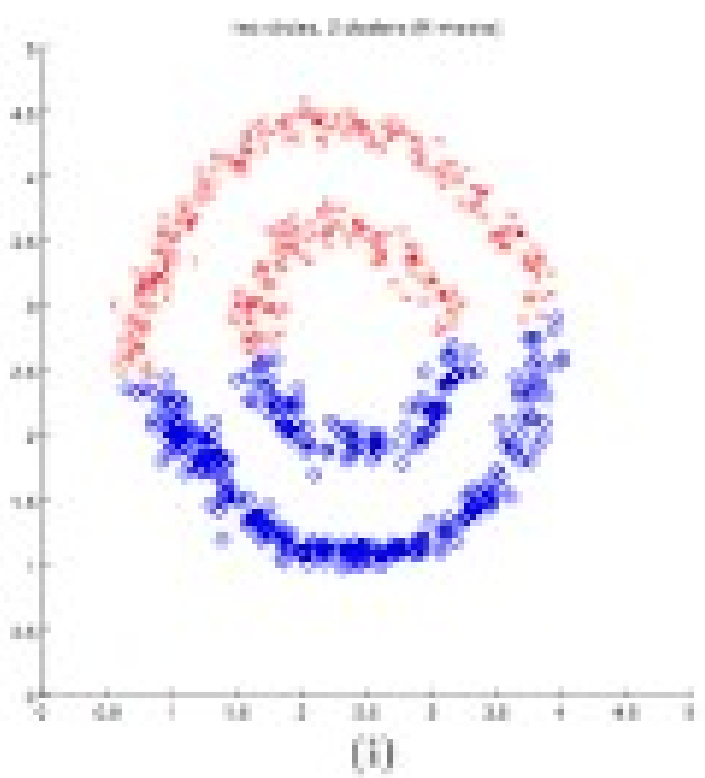
$$(D - W) y = \lambda D y$$

- Setting $x = D^{1/2} y$ $D^{-1/2} (D - W) D^{-1/2} x = \lambda x$

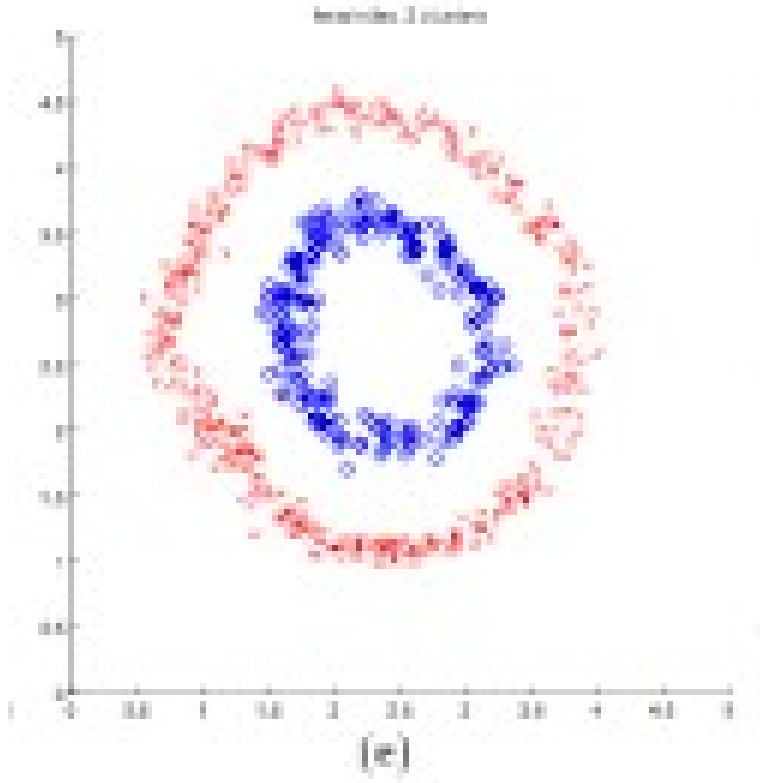
- Eigenvalues of the Laplacian are approximate solutions to mininum normalized cut problem.
  - The lowest eigenvalue is 0, eigenvector is
  - The second lowest contains the solution
    - The corresponding eigenvector contains the cluster indicator for each data point


NIP
2001
(a)
(b)
(c)
(d)
(e)
(f)

K-means Vs. normalized cuts



K-means



Spectral Clustering